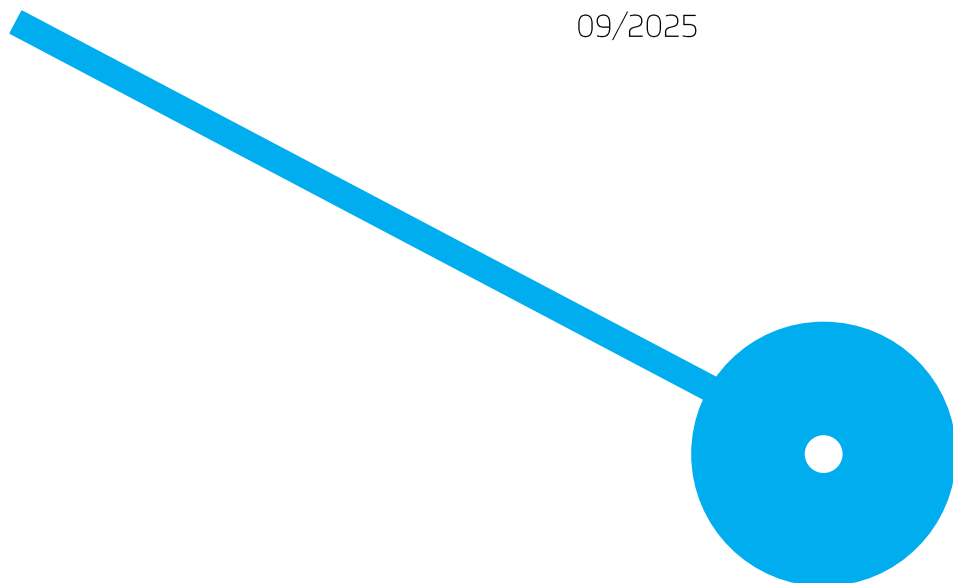


Sistema de Recomendação com Integração de Explicabilidade

Guilherme Francisco Fernandes Oliveira Dias Mieiro

09/2025



Sistema de Recomendação com Integração de Explicabilidade

Guilherme Francisco Fernandes Oliveira Dias Mieiro
8180148

Orientador

Professora Doutora Mariana Reimão Carvalho

Relatório de Projeto apresentado para cumprimento dos requisitos necessários à obtenção do grau de Licenciado em Engenharia Informática pela Escola Superior de Tecnologia e Gestão do Instituto Politécnico do Porto.

Declaração de Integridade

Eu, Guilherme Francisco Mieiro, estudante nº 8180148, da Licenciatura de Engenharia Informática da Escola Superior de Tecnologia e Gestão do Instituto Politécnico do Porto, declaro que não fiz plágio nem auto-plágio, pelo que o trabalho intitulado Sistema de Recomendação com Integração de Explicabilidade é original e da minha autoria, não tendo sido usado previamente para qualquer outro fim. Mais declaro que todas as fontes usadas estão citadas, no texto e na bibliografia final, segundo as regras de referência adotadas na instituição.

Índice

Índice	4
Índice de Figuras.....	6
Índice de Tabelas	7
Agradecimentos	8
Lista de Abreviaturas e Siglas	9
CAP I – Contextualização e Motivação	10
1. Introdução.....	10
1.1 Objetivos.....	10
1.2 Resultados esperados	10
1.3 Organização do documento	11
2. Fundamentação teórica	11
2.1 Revisão da Literatura.....	11
2.2 Sistemas de Recomendação	13
2.3 Tipos de Sistemas de Recomendação.....	13
2.4 Avaliação de Sistemas de Recomendação	15
2.5 Técnicas de Explicabilidade (XAI).....	17
CAP II – Conceptualização do problema/projeto	18
3. Requisitos	18
3.1 Atores e Casos de Uso	18
3.2 Requisitos Funcionais	19
3.3 Requisitos Não Funcionais.....	21
4. Arquitetura Conceptual.....	22
CAP III – Metodologia de operacionalização do trabalho	24
5. Processo e Metodologia de trabalho.....	24
5.1 Planeamento e organização do trabalho	24
5.2 Ferramentas Utilizadas no Desenvolvimento	25
6. Desenvolvimento da solução	26
6.1 Arquitetura Geral do Sistema	26
6.2 Preparação e Processamento de Dados.....	27

6.3 Implementação dos Modelos de Recomendação	30
6.4 Avaliação e Análise de Desempenho dos Modelos	33
6.5 Previsão das Recomendações	33
6.6 Explicabilidade das Recomendações	33
6.7 Interface Interativa com <i>Streamlit</i>	35
6.8 Geração de <i>Logs</i>	42
CAP IV – Discussão dos resultados	43
7. Apresentação e discussão dos resultados.....	43
8. Apresentação e discussão dos impedimentos e/ou constrangimentos	47
CAP V – Conclusão.....	48
9. Reflexão crítica dos resultados	48
10. Conclusão e trabalho futuro.....	48
11. Referências	49

Índice de Figuras

Figura 1- Diagrama de Atores e Use Cases.....	18
Figura 2- Arquitetura Conceptual.....	23
Figura 3 - Diagrama de Gantt	25
Figura 4- Arquitetura Geral do Sistema.....	27
Figura 5 - Estrutura dos dados nos ficheiros do MovieLens	28
Figura 6 - Filtragem e Amostragem dos dados para CBF	28
Figura 7 - Filtragem e Amostragem dos dados para CF.....	28
Figura 8 - Pré-processamento dos Géneros para CBF	29
Figura 9 - Divisão dos dados em dataframes rated e unrated.....	30
Figura 10 - Cálculo do vetor de utilizador em filtragem híbrida	32
Figura 11 - Explicações baseadas em vizinhos.....	34
Figura 13- Gráfico de barras de valores SHAP	35
Figura 12- Gráfico circular de valores SHAP	35
Figura 14- Listagem dos itens recomendados.....	36
Figura 15 - Detalhes de um item recomendado.....	36
Figura 16 - Explicações locais baseadas em vizinhos	37
Figura 17 - Explicações locais SHAP	37
Figura 18 - Seleção de perfil de utilizador	38
Figura 19 - Explicações globais SHAP	38
Figura 20 - Seleção de modelo e tipo de filtragem.....	39
Figura 21 - Análise e avaliação do modelo selecionado (learning curve).....	39
Figura 22- Análise e avaliação do modelo selecionado (cross validation).....	39
Figura 23 - Adição de novo perfil de utilizador.....	40
Figura 24 - Adição de novo item.....	40
Figura 25 - Adição de nova interação.....	41
Figura 26 - Média das métricas no conjunto de treino (validação cruzada com 3 folds).....	45
Figura 27 - Média das métricas no conjunto de validação (validação cruzada com 3 folds). 46	

Índice de Tabelas

Tabela 1- Requisito Funcional 1	19
Tabela 2 - Requisito Funcional 2	19
Tabela 3 - Requisito Funcional 3	19
Tabela 4 - Requisito Funcional 4	20
Tabela 5 - Requisito Funcional 5	20
Tabela 6 - Requisito Funcional 6	20
Tabela 7 - Requisito Funcional 7	20
Tabela 8 - Requisito Não Funcional 1	21
Tabela 9 - Requisito Não Funcional 2	21
Tabela 10 - Requisito Não Funcional 3	22
Tabela 11 - Requisito Não Funcional 4	22
Tabela 12 - Parâmetros de treino de rede neuronal (cbf)	43
Tabela 13 - Parâmetros de treino de rede neuronal (cf)	43
Tabela 14 - Parâmetros de treino de rede neuronal (híbrida)	44
Tabela 15 - Parâmetros treino de random forest (cbf)	44
Tabela 16 - Parâmetros treino de random forest (híbrida).....	44
Tabela 17 - Parâmetros treino de xgboost (cbf)	44
Tabela 18 - Parâmetros treino de xgboost (híbrida)	45

Agradecimentos

Primeiramente, gostaria de agradecer à minha família, por todo o apoio e ajuda em criar um ambiente estável e propício ao meu desenvolvimento enquanto pessoa e estudante.

Gostaria também de agradecer à Professora Doutora Mariana, por ser a minha orientadora durante este projeto e por todas as orientações dadas no decorrer do mesmo.

Por fim, mas não menos importante, gostaria de agradecer a todos os professores da Escola Superior de Tecnologia e Gestão, que, ao longo do curso contribuíram para o meu progresso e sucesso académico.

Lista de Abreviaturas e Siglas

CBF	Content-Based Filtering
CF	Collaborative Filtering
SHAP	SHapley Additive exPlanations
LIME	Local Interpretable Model-Agnostic Explanations
XGBoost	Extreme Gradient Boost
IA	Inteligência Artificial
ML	Machine Learning
SVD	Singular Value Decomposition
CAM	Class Activation Map
XAI	Explainable Artificial Intelligence
MRS	Music Recommendation System
RS	Recommendation System
ES	Expert System
DL	Deep Learning
HRS	Hybrid Recommendation System

CAP I – Contextualização e Motivação

1. Introdução

Com o avanço tecnológico e a massificação dos serviços online, os utilizadores tem, agora, acesso a uma crescente quantidade de informação. É assim que surge o chamado problema de “data overloading” (sobrecarga de informação), isto é, existe tanta informação que o utilizador deixa de ser capaz de a processar toda eficazmente. Torna-se então essencial a aplicação de mecanismos de filtragem que selecionem e priorizem os conteúdos mais relevantes para cada indivíduo assim nascem os algoritmos de recomendação [1].

O presente projeto, visa desenvolver um sistema de recomendação com integração de explicabilidade, que, recorrendo a algoritmos de *machine learning*, não só gere recomendações, mas também que justifique e fundamente o porquê de fazer ditas recomendações, promovendo assim mais confiança e credibilidade sobre as mesmas.

1.1 Objetivos

O principal objetivo deste projeto é a implementação de um sistema de recomendação com um módulo de explicabilidade integrado. Assim é suposto o sistema ser capaz de recomendar itens a utilizadores com base em diferentes fatores além disso, o sistema deve ainda ser capaz de explicar e fundamentar o porquê de ter feito dadas recomendações.

Com vista a alcançar o objetivo principal, são definidos os seguintes subobjetivos:

- Recolher e tratar os dados relevantes para treinar os modelos de recomendação;
- Implementar os diferentes algoritmos de recomendação (baseados em filtragem colaborativa, conteúdo e híbridos) e treinar os modelos associados;
- Avaliar a performance dos modelos desenvolvidos, recorrendo às métricas apropriadas;
- Integrar o módulo de explicabilidade com as técnicas, adequadas a cada contexto;
- Desenvolver uma interface gráfica que permita a apresentação e interação com o sistema e as suas funcionalidades.

1.2 Resultados esperados

Com o desenvolvimento deste projeto, espera-se no final entregar um sistema de recomendação funcional que materialize os objetivos previamente definidos.

De entre os resultados esperados destacam-se:

- Um *pipeline* de recolha e tratamento de dados;
- Diferentes modelos de recomendação, devidamente validados com uma boa capacidade preditiva, ou pelo menos, com uma capacidade preditiva em conformidade com a esperada, tendo em conta as diferentes “forças” e limitações de cada abordagem de recomendação;
- Disponibilização de recomendações personalizadas acompanhadas das respetivas explicações;

- Aquisição de um maior entendimento e compreensão das diferentes abordagens de recomendação, das peculiaridades de cada uma e como as mesmas influenciam a experiência do utilizador (um “olhar por detrás da cortina”).

1.3 Organização do documento

Este relatório está dividido em cinco capítulos:

- Capítulo I – Contextualização e Motivação: Inclui uma descrição geral e de alto nível do projeto, bem como a apresentação dos objetivos do mesmo. São apresentados os resultados esperados do projeto e é feita uma contextualização dos diferentes conceitos e fundamentos relacionados com o tema;
- Capítulo II – Contextualização do Problema/Projeto: Neste capítulo, com a definição dos requisitos do sistema, são detalhados os aspetos mais técnicos do problema. Também aqui, posteriormente, é definida a arquitetura conceptual da solução proposta;
- Capítulo III – Metodologia de Operacionalização do Trabalho: Descreve o processo seguido para o desenvolvimento da solução, a metodologia adotada e as etapas concretas de implementação;
- Capítulo IV – Discussão dos Resultados: Inclui uma análise dos resultados obtidos e uma discussão dos principais impedimentos ou constrangimentos verificados durante o projeto;
- Capítulo V – Conclusão: Apresenta uma reflexão crítica sobre o trabalho realizado e dos resultados obtidos.

2. Fundamentação teórica

2.1 Revisão da Literatura

Nesta secção, é feita uma breve revisão de artigos relacionados com o tema, com o intuito de apresentar uma visão geral do mesmo, nomeadamente, das técnicas usadas, peculiaridades e limitações das mesmas.

O artigo [2] oferece uma visão geral dos sistemas de recomendação, destacando sua evolução desde 1995 e o seu papel em personalizar experiências em *e-commerce*, como *Amazon* e *Netflix*. Explora técnicas de IA como *machine learning* e *data mining*, para enfrentar desafios como sobrecarga de informação.

Em [3], destacando a sua importância na filtragem de informações em meio à expansão da internet, o artigo revê avanços em sistemas de recomendação. Analisa algoritmos de filtragem colaborativa e baseada em conteúdo, discutindo desafios como a esparsidade dos dados e *cold-start*.

Destacando a combinação de filtragem colaborativa com outras técnicas, como baseada em conteúdo, em [4], o artigo revê diferentes sistemas híbridos de recomendação da última década, para superar problemas de *cold-start* e esparsidade. Analisa 48 estudos, identificando filmes como domínio principal, sugerindo desafios futuros como contextualização e uso de *datasets* maiores.

O artigo [5] revê a evolução dos sistemas de recomendação baseados em filtragem colaborativa (*CF*), abordando métodos tradicionais como *SVD* e avanços com abordagens híbridas. Destaca desafios como a esparsidade dos dados e a dimensionalidade em *big data*.

É proposto em [6], um sistema híbrido de recomendação para livros que combina filtragem colaborativa e baseada em conteúdo, dessa forma, superando limitações como *cold start* e escalabilidade. Divide-se em três fases: identificação de usuários similares, seleção de itens candidatos via vetores de perfil e predição com a equação de *Resnick*, mostrando superioridade sobre métodos tradicionais em testes.

São avaliadas as técnicas de *Explainable AI*, *SHAP*, *LIME* e *CAM*, em [7], comparando a eficácia de cada uma a interpretar modelos de *machine learning*. Destaca a aplicação destas técnicas em diversas áreas, como saúde e finanças.

São exploradas a integração de *XAI* em sistemas de recomendação, em [8], destacando como explicações aumentam a confiança do utilizador e a eficácia das recomendações. Analisa métodos como *LIME* e *SHAP*, enfatizando sua aplicação em domínios como *e-commerce* e *streaming*.

Em [9], o artigo explora como a explicabilidade pode melhorar sistemas de recomendação musical (*MRS*), destacando a integração de *Explainable AI* para aumentar confiança e tolerância do utilizador em recomendações inesperadas. Analisa dimensões de *XAI* adaptadas ao contexto musical, sugere formas de integrar explicações em *MRS* e discute desafios de avaliação.

O artigo [10], compara as arquiteturas de *Expert Systems (ES)*, *Recommendation Systems (RS)* e *Explainable AI (XAI)*, destacando a evolução desta no contexto da *IA*. Apresenta *ES* como baseados em regras com desafios de aquisição de conhecimento, *RS* como filtros personalizados usando *ML/DL* e *XAI* como uma solução para transparência de modelos de caixa-preta.

É proposto um modelo preditivo para diabetes, em [11], usando *XAI* com a técnica de explicabilidade *LIME*, integrando variáveis como glicose e IMC (índice de massa corporal), com testes em quatro algoritmos (tendo o melhor uma precisão de 93%). Destaca a recomendação de alimentos saudáveis para a prevenção da diabetes e aborda problemas como o de *cold start*.

Em [1], o artigo oferece uma revisão sistemática de sistemas de recomendação e analisa a aplicação dos mesmos em áreas como *e-commerce* e *e-learning*, com foco em desafios como escalabilidade e *cold-start*. Avalia técnicas de filtragem colaborativa, baseada em conteúdo e *deep learning*.

O artigo [12], analisa como explicações em sistemas de recomendação aumentam a confiança do utilizador, abordando a evolução de *Explainable AI (XAI)* desde sistemas baseados em regras (*Expert Systems*) até redes neurais. Destaca a importância da avaliação centrada no humano e a conformidade com o *GDPR (General Data Protection Regulation)*.

Em [13], o artigo compara as técnicas *LIME* e *SHAP* na recomendações de filmes com alta esparsidade, revelando que *SHAP* supera *LIME* em dados esparsos devido à menor variância, enquanto *LIME* é melhor em dados densos.

O artigo [14], apresenta uma taxonomia detalhada de métodos de *Explainable Ai (XAI)*, baseada em mais de 50 surveys, para guiar pesquisadores e programadores na escolha destes métodos. Destaca traços como portabilidade (modelo agnóstico ou não) e localidade (explicações globais ou específicas), ilustrando exemplos como *LIME* e *SHAP*.

É apresentada uma abordagem híbrida que combina filtragem colaborativa (*CF*) e baseada em conteúdo (*CBF*), em [15], para melhorar sistemas de recomendação. Integra dados de interações do utilizador (*CF*) com atributos de itens (*CBF*), superando assim limitações como *cold-start* e falta de diversidade.

Em [16], o artigo explora os métodos de *Explainable Ai SHAP* e *LIME*, focando em como diferentes modelos e a colinearidade dos dados afetam explicações em dados tabulares. Com este propósito é utilizado um estudo de casos de infarto do miocárdio. Destaca que ambas as técnicas são influenciadas por colinearidade dos dados e pela escolha do modelo, sugerindo cautela na interpretação.

O artigo [17] propõe um modelo baseado em *Explainable AI (XAI)*, usando redes neurais, para recomendar culturas com base no solo e condições ambientais. Destaca uma melhoria de 12% na precisão em relação a outros métodos, oferecendo transparência para agricultores.

São revistos sistemas híbridos de recomendação (*HRS*) em *e-commerce*, em [18], analisando os avanços dos últimos seis anos. Combina filtragem colaborativa e baseada em conteúdo, para superar problemas de esparsidade e *cold-start*, destacando melhorias na experiência de utilizador e vendas.

2.2 Sistemas de Recomendação

De acordo com [19], sistemas de recomendação são “ferramentas de software e técnicas que fornecem a utilizadores específicos, itens que são provavelmente do seu interesse”. Estando os utilizadores expostos a uma quantidade ilimitada de informação e possuindo os mesmos uma capacidade limitada de a processar eficazmente, os sistemas de recomendação funcionam como filtros inteligentes, tentando prever os itens que mais provavelmente serão do interesse de cada utilizador [20]. Estes sistemas têm sido amplamente usados em diversas áreas como comercio online, saúde, educação, procura de emprego, entre outras [1].

2.3 Tipos de Sistemas de Recomendação

Podemos dividir os sistemas de recomendação em três principais categorias, filtragem baseada em conteúdo, filtragem colaborativa e sistemas híbridos (a combinação das 2 abordagens anteriores) [1].

2.3.1 Filtragem Baseada em Conteúdo (*Content-Based Filtering*)

Em filtragem baseada em conteúdo, o sistema tenta recomendar itens com características semelhantes aos que o utilizador gostou no passado. Por exemplo, se o utilizador sistematicamente avaliou positivamente filmes do género comédia, o sistema tenderá a recomendar mais filmes desse género no futuro [19].

Este método exige que os itens estejam bem descritos (através de *features*/características), sendo a qualidade dessas *features* essencial para a geração de boas recomendações [2].

Assim, podemos apontar as seguintes vantagens:

- Independência de dados de outros utilizadores: Sendo as recomendações baseadas nas características de itens que o próprio utilizador avaliou/interagiu no passado, o sistema não

precisa de informações/perfis relativos a outros utilizadores, funcionando bem, mesmo em ambientes com poucos utilizadores [1].

- Recomendações para itens novos (*Cold-Start* de item): O sistema não sofre do chamado problema de “*cold-start*” relativo a itens novos, que ainda não foram avaliados por utilizadores, uma vez que independentemente de o item ser novo ou não, desde que adequadamente descrito, a sua similaridade a outros itens pode na mesma ser calculada [1].
- Capacidade de explicação: Os sistemas de recomendação baseados em conteúdos podem fornecer uma lista das características do item que clarificam o porquê de o mesmo ter sido recomendado, isso ajuda a aumentar a confiança e credibilidade do sistema no utilizador [20].

Da mesma forma, podemos notar as seguintes limitações:

- Falta de diversidade/ Sobre especialização: O sistema tende a recomendar itens semelhantes aos já consumidos, isto pode levar pouca variedade nos itens recomendados [15] [6].
- Dependência de metadados/*Features* de qualidade: Requer uma boa representação dos itens através de atributos bem definidos e estruturados [4] [6].
- Dificuldade em capturar gostos complexos ou implícitos, que podem não estar diretamente refletidos nas *features* dos itens [15].
- Ignora aspetos sociais ou colaborativos: Este sistema ignora aspetos sociais como recomendações baseadas em utilizadores semelhantes ou tendências sociais [3] [15].

2.3.2 Filtragem Colaborativa (*Collaborative Filtering*)

Em filtragem colaborativa, parte-se do princípio de que utilizadores com preferências similares no passado, tenderão a ter preferências similares no futuro.

Assim, com base no histórico da interação de diferentes utilizadores com itens (ignorando o conteúdo de cada item) o sistema tenta recomendar itens com os quais o utilizador ainda não interagiu, mas outros utilizadores com preferências semelhantes às dele, interagiram positivamente [15] [2] [8] [5].

Existem duas variantes principais:

- *User-based*: recomenda itens que utilizadores semelhantes (vizinhos) gostaram [5].
- *Item-based*: recomenda itens semelhantes aos que o utilizador gostou no passado, com base nas preferências de todos os utilizadores (as “*features*” do item não são derivadas do seu conteúdo, mas sim das interações de diferentes utilizadores com o mesmo) [5].

Assim, podemos apontar as seguintes vantagens:

- Não depende de metadados dos itens: Sendo as recomendações baseadas nas interações entre utilizadores e itens, este sistema não necessita de conhecimento explícito sobre o conteúdo dos itens [15].
- Pode revelar padrões inesperados (descoberta de novos interesses): Ao contrário do sistema baseado em conteúdo, que tende a recomendar itens semelhantes ao que o utilizador já

conhece, o sistema colaborativo recomenda itens que outros utilizadores com gostos semelhantes interagiram positivamente, não sendo estes itens necessariamente semelhantes aos habitualmente consumidos pelo utilizador em questão, isto permite ao utilizador descobrir novos interesses [19] [1].

Da mesma forma, podemos notar as seguintes limitações:

- Problemas com dados esparsos (poucas avaliações): Este problema surge do facto de os utilizadores por norma avaliarem apenas uma pequena quantidade dos itens existentes. Isto pode levar a um número insuficiente de avaliações comuns entre objetos ou utilizadores, causando grandes desvios no cálculo da similaridade e afetando negativamente a qualidade das recomendações [15] [4].
- Fraco desempenho em situações de *cold start* (utilizadores ou itens novos): No caso de novos utilizadores, o sistema não tem histórico de preferências para encontrar utilizadores semelhantes. Já no caso de itens novos, o sistema tem dificuldade em recomendá-los uma vez que estes ainda não foram avaliados por um número grande o suficiente de utilizadores [5] [4].
- Problema “*Long-Tail*” (Viés de popularidade): Os sistemas de filtragem colaborativa, muitas vezes se deparam com o chamado problema “*long-tail*”, isto é uma situação onde uma grande parte das interações dos utilizadores, se referem apenas a um pequeno grupo de itens populares, enquanto os restantes desfrutam de pouca ou nenhuma interação [4] [15].

2.3.3 Sistemas Híbridos

Sistemas híbridos integram 2 ou mais técnicas com o objetivo de, usando as “forças” de cada técnica, mitigar as limitações de cada uma quando usadas de forma isolada. Isto, normalmente, resulta em recomendações mais precisas [20] [1] [4].

Existem várias formas de integração, entre as quais:

- Agregação de resultados: As listas de recomendações geradas por diferentes técnicas, são combinadas, para formar uma lista final [4].
- Modelos sequenciais: Uma técnica é usada para gerar uma lista de recomendações iniciais e é posteriormente aplicada, outra técnica, sobre essa mesma lista, para a refinar [4].
- Modelos integrados: Algoritmos incorporam, diretamente, tanto os dados das interações dos utilizadores como as características dos itens [1] [15] [3].

Este tipo de sistema é particularmente útil quando existe pouca informação disponível sobre utilizadores ou itens, uma vez que permite combinar diferentes fontes de dados para gerar recomendações mais completas e relevantes.

2.4 Avaliação de Sistemas de Recomendação

A avaliação de sistemas de recomendação é uma etapa fundamental para medir a qualidade das sugestões geradas e validar a eficácia dos algoritmos utilizados. Sem uma avaliação rigorosa, torna-

se difícil compreender se o sistema está, de facto, a fornecer recomendações úteis e relevantes para os utilizadores.

Existem três principais tipos de avaliação, *user studies*, avaliação *online* e avaliação *offline* [8].

2.4.1 User Studies

Em *user studies*, é pedido aos utilizadores que interajam com o sistema de recomendação e posteriormente deem o seu *feedback*. Isto tem a vantagem de receber o *feedback* diretamente dos utilizadores, contudo é por vezes complicado de fazer com uma quantidade grande o suficiente de utilizadores de forma a garantir que os mesmos representam adequadamente a população [8].

2.4.2 Avaliação online

A avaliação *online* é realizada com utilizadores reais em ambiente de produção, através de testes A/B ou análise de métricas de *engagement* (ex.: cliques, tempo de visualização). Ao contrário dos *user studies* em que os utilizadores sabem que estão a ser avaliados, na avaliação *online* os utilizadores interagem com o sistema de forma natural, sem estarem cientes de que o seu comportamento está a ser monitorizado para fins de avaliação [8].

A principal desvantagem deste tipo de avaliação é que, para ser implementado eficazmente, é necessário que já exista um grande número de utilizadores registados. Por esse motivo, torna-se difícil aplicar este método durante as fases iniciais de desenvolvimento, quando a base de utilizadores ainda é limitada [8].

2.4.3 Avaliação offline

Consiste em utilizar dados históricos (como *datasets* do *MovieLens*) para simular recomendações e posteriormente compará-las com os resultados reais conhecidos [4].

Este tipo de avaliação tem a vantagem de, contrariamente às avaliações online e *user studies*, não depender de utilizadores registados no sistema [8].

Para isso, algumas métricas são utilizadas, como:

- **RMSE (Root Mean Squared Error):** Mede o erro médio quadrático entre as classificações previstas pelo modelo e os valores reais. Penaliza os erros maiores, sendo sensível a *outliers* [3].

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(y_i - \hat{y}_i)^2}{n}}$$

- **MAE (Mean Absolute Error):** Mede o erro absoluto médio entre as previsões do modelo e os valores reais. Todos os erros têm o mesmo peso, tornando-o mais robusto a *outliers* do que o RMSE [3].

$$MAE = \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{n}$$

De notar que estas métricas são usadas para problemas de regressão (o sistema tenta prever 1 variável contínua), outras métricas como *precision*, *recall* ou *F1-score* também são utilizadas, mas em problemas de classificação, isto é, em sistemas onde se pretende prever uma variável discreta que assume um valor de entre um conjunto de opções previamente definidas (por exemplo, uma enumeração de categorias ou classes) [1].

2.5 Técnicas de Explicabilidade (XAI)

A explicabilidade em sistemas de recomendação permite aumentar a confiança dos utilizadores nas sugestões geradas e facilita a compreensão dos critérios que levaram a dada recomendação. Assim, estas técnicas, ajudam a tornar os modelos mais transparentes, interpretáveis e justificáveis, promovendo uma interação mais informada entre o sistema e o utilizador [16].

Existem várias abordagens para a explicação de recomendações, incluindo:

- Explicações baseadas em conteúdo (*feature-based explanations*): Destacam as características do item que justificam a recomendação, como o género, autor ou características específicas que o utilizador valorizou anteriormente [9].
- Explicações baseadas em vizinhos (*user explanations*): Indicam que a recomendação foi feita porque outros utilizadores semelhantes gostaram daquele item [9].
- Modelos explicáveis por design: Utilizam algoritmos intrinsecamente interpretáveis, que permitem compreender facilmente os fatores que influenciam a recomendação [9].
- Técnicas pós-hoc: Aplicam métodos externos para interpretar modelos complexos (como redes neurais), por exemplo, utilizando ferramentas como *SHAP* (*SHapley Additive exPlanations*) ou *LIME* (*Local Interpretable Model-agnostic Explanations*) para analisar a importância das variáveis [9].

Em [13] ficamos a saber que a escolha entre SHAP e LIME depende do contexto, dos requisitos da aplicação e das características dos dados:

Para explicações locais rápidas e em dados com alta densidade, LIME pode ser uma opção eficaz.

Quando são necessárias explicações globais (para além das locais) e em cenários com dados esparsos, *SHAP* ou as suas variantes (como *CLIMB*) tendem a ser superiores [13].

CAP II – Conceptualização do problema/projeto

3. Requisitos

3.1 Atores e Casos de Uso

Para o problema em questão, foi identificado um único ator, denominado de “utilizador do sistema”.

- Utilizador do Sistema – Pessoa que através da interface de utilizador (apenas existe “frontoffice”), interage com as diferentes funcionalidades do sistema.

O ator e as relações descritas, encontram-se ilustradas na figura 1.

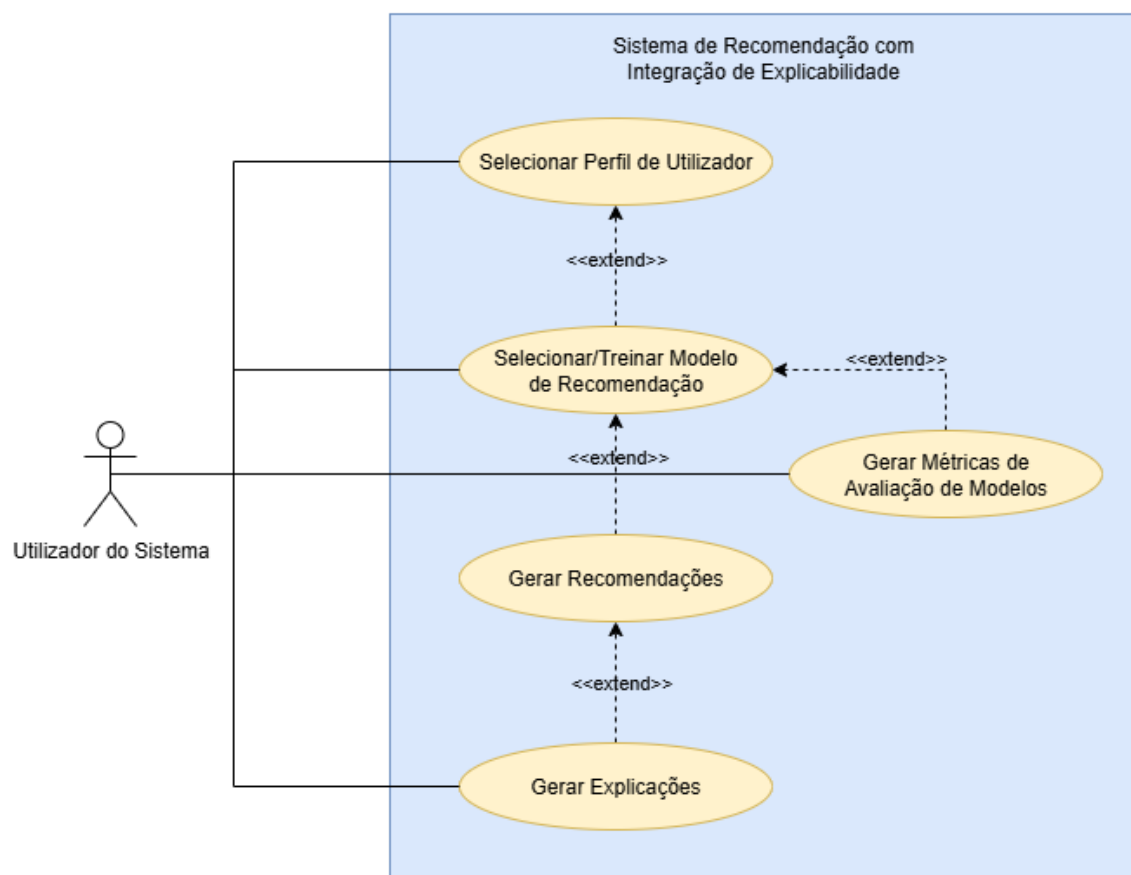


Figura 1- Diagrama de Atores e Use Cases

3.2 Requisitos Funcionais

Os requisitos funcionais especificam o comportamento do sistema, descrevendo as funções que este deve desempenhar para responder às necessidades dos utilizadores.

Id	RF1
Nome	Importação e tratamento de dados
Descrição	O sistema deve ser capaz de carregar e pré-processar os dados necessários para o funcionamento dos algoritmos de recomendação.
Prioridade	Alta
Restrições	Necessário haver dados de importação devidamente organizados em um formato compatível.

Tabela 1- Requisito Funcional 1

Id	RF2
Nome	Treino de modelos de recomendação
Descrição	O sistema deve permitir treinar modelos de recomendação baseados em filtragem colaborativa, baseada em conteúdo e híbrida, com diferentes configurações.
Prioridade	Alta
Restrições	Necessário haver dados adequados para o treino dos modelos.

Tabela 2 - Requisito Funcional 2

Id	RF3
Nome	Geração de recomendações personalizadas
Descrição	O sistema deve ser capaz de gerar recomendações de itens personalizadas a um utilizador específico, com base no histórico de interações (entre outros dados).
Prioridade	Alta
Restrições	Necessário haver modelos treinados e dados sobre os quais os modelos possam efetuar previsões.

Tabela 3 - Requisito Funcional 3

Id	RF4
Nome	Apresentação de explicações para as recomendações

Descrição	O sistema deve apresentar explicações que fundamentem e justifiquem cada recomendação feita, recorrendo a técnicas de explicabilidade.
Prioridade	Alta
Restrições	Necessário haver modelos treinados e recomendações feitas pelos mesmos.

Tabela 4 - Requisito Funcional 4

Id	RF5
Nome	Interface gráfica interativa
Descrição	O sistema deve disponibilizar uma interface gráfica simples e intuitiva, que permita interagir com o sistema, explorando as recomendações e suas explicações.
Prioridade	Média
Restrições	

Tabela 5 - Requisito Funcional 5

Id	RF6
Nome	Avaliação e comparação dos diferentes modelos
Descrição	Uma avaliação e comparação dos diferentes modelos deve ser realizada e os dados relevantes apresentados, utilizando as métricas adequadas.
Prioridade	Média
Restrições	Necessário haver modelos treinados.

Tabela 6 - Requisito Funcional 6

Id	RF7
Nome	Seleção de utilizadores
Descrição	O sistema deve permitir ao utilizador selecionar diferentes perfis de utilizador para os quais pretende obter recomendações.
Prioridade	Média
Restrições	Necessário haver dados relativos a múltiplos perfis de utilizador.

Tabela 7 - Requisito Funcional 7

3.3 Requisitos Não Funcionais

Os requisitos não funcionais definem as propriedades que garantem a qualidade do sistema, como desempenho, usabilidade e organização do código.

Id	RNF1
Nome	Usabilidade
Descrição	A interface gráfica deve ser intuitiva, clara e de fácil utilização, permitindo ao utilizador interagir com o sistema sem necessidade de conhecimentos técnicos avançados.
Prioridade	Alta
Critério de aceitação	Através da interface o utilizador deve poder: • Selecionar perfil de utilizador; • Selecionar tipo de modelo e filtragem; • Ver as recomendações feitas; • Ver a explicação das recomendações feitas; Caso conceitos técnicos que não façam parte do conhecimento da pessoa comum sejam apresentados, estes devem vir acompanhados de uma mensagem que descreva o mesmo.

Tabela 8 - Requisito Não Funcional 1

Id	RNF2
Nome	Modularidade e organização do código
Descrição	O sistema deve ser desenvolvido de forma modular, com separação clara entre funções responsáveis por diferentes tarefas (leitura de dados, treino de modelos, visualização, etc.), facilitando a manutenção e futuras extensões.
Prioridade	Alta
Critério de aceitação	O sistema está dividido de forma modular com clara separação entre funções responsáveis por diferentes tarefas.

Tabela 9 - Requisito Não Funcional 2

Id	RNF3
Nome	Portabilidade
Descrição	A aplicação deve poder ser executada em diferentes ambientes com <i>Python</i> instalado, idealmente com suporte a ambientes virtuais para facilitar a instalação das dependências.
Prioridade	Alta
Critério de aceitação	O sistema tem o ficheiro “requirements.txt” ou outro com o mesmo propósito, que lista todas as dependências do programa.

Tabela 10 - Requisito Não Funcional 3

Id	RNF4
Nome	Reprodutibilidade
Descrição	Os resultados devem ser reprodutíveis, dentro dos limites impostos por algoritmos com componentes aleatórios. Sempre que possível, o sistema deve permitir obter resultados consistentes ao repetir o processo com os mesmos dados, parâmetros e ambiente de execução.
Prioridade	Média
Critério de aceitação	Sempre que permitido pelo contexto, parâmetros como “random_state” ou outros com propósito semelhante são usados.

Tabela 11 - Requisito Não Funcional 4

4. Arquitetura Conceptual

Como referido anteriormente, para facilitar sua manutenção e expansão, o sistema será desenvolvido de forma modular. Nesta secção será apresentada a arquitetura conceptual da solução, isto é, será feita uma descrição de alto nível dos diferentes componentes/módulos que compõe a solução e como eles interagem entre si para resolver o problema proposto.

- Módulo de Dados: É responsável pela leitura e tratamento/transformação de dados;
- Módulo de Modelos: Consome os dados transformados e com eles treina modelos de recomendação;
- Módulo de Avaliação: Consome os modelos de recomendação treinados e avalia o seu desempenho utilizando as métricas adequadas (e gera os dados correspondentes);

- Módulo de Previsão: Consome os modelos de recomendação treinados, bem como os dados transformados adequados e com os mesmos gera previsões e consequentemente recomendações;
- Módulo de Explicabilidade: Consome as recomendações feitas, o modelo usado para as fazer e dados de interações históricas e com eles gera explicações, fundamentando e justificando as recomendações feitas;
- Interface Gráfica: Consome as recomendações, explicações e dados relativos à avaliação de modelos, apresentando estes para o utilizador.

Na imagem 2 é ilustrada a arquitetura conceptual descrita.

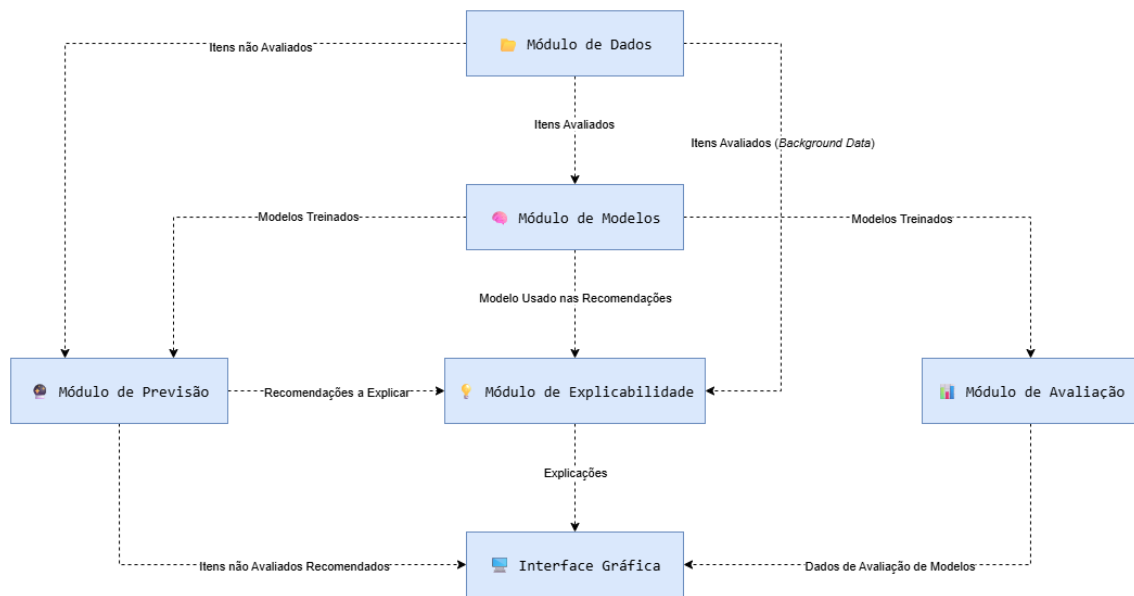


Figura 2- Arquitetura Conceptual

CAP III – Metodologia de operacionalização do trabalho

5. Processo e Metodologia de trabalho

Nesta secção serão descritos os processos e as abordagens sistemáticas usadas ao longo do desenvolvimento do projeto, bem como as ferramentas utilizadas.

5.1 Planeamento e organização do trabalho

Antes de iniciar o desenvolvimento propriamente dito do projeto, foi realizada uma fase exploratória inicial dedicada à pesquisa e estudo do tema. Durante este período foram consultados artigos científicos e outras fontes relevantes com vista a obter uma compreensão sólida dos conceitos teóricos envolvidos no trabalho.

Posteriormente, foram investigadas e decididas as ferramentas e tecnologias a serem utilizadas bem como realizada a conceptualização da solução.

Por fim, foi adotada uma abordagem prática, centrada na experimentação e validação contínua das decisões tomadas, resultando na implementação incremental dos diferentes componentes do sistema.

Assim, podemos organizar o trabalho nas seguintes fases distintas, mas interligadas:

1. Recolha e tratamento de dados: Nesta fase, partindo dos *datasets* iniciais, são aplicadas as transformações, para obter os dados necessários, no formato adequado, ao treino do modelo de recomendação em questão;
2. Implementação do modelo: Nesta fase o modelo em questão é implementado;
3. Implementação da explicabilidade: Nesta fase as técnicas de explicabilidade adequadas ao tipo de filtragem em questão são implementadas;
4. Avaliação e análise do modelo: Nesta fase é feita a avaliação dos modelos desenvolvidos, bem como a implementação das ferramentas necessárias para o fazer;
5. Implementação da interface gráfica: Nesta fase é implementada a interface gráfica para mais facilmente interagir com o sistema e visualizar os resultados do mesmo.

De notar que, como ilustrado na figura 3, estas fases, repetiram-se de forma sequencial para cada um dos tipos de recomendação implementados (filtragem baseada em conteúdo, colaborativa e, por fim, híbrida).

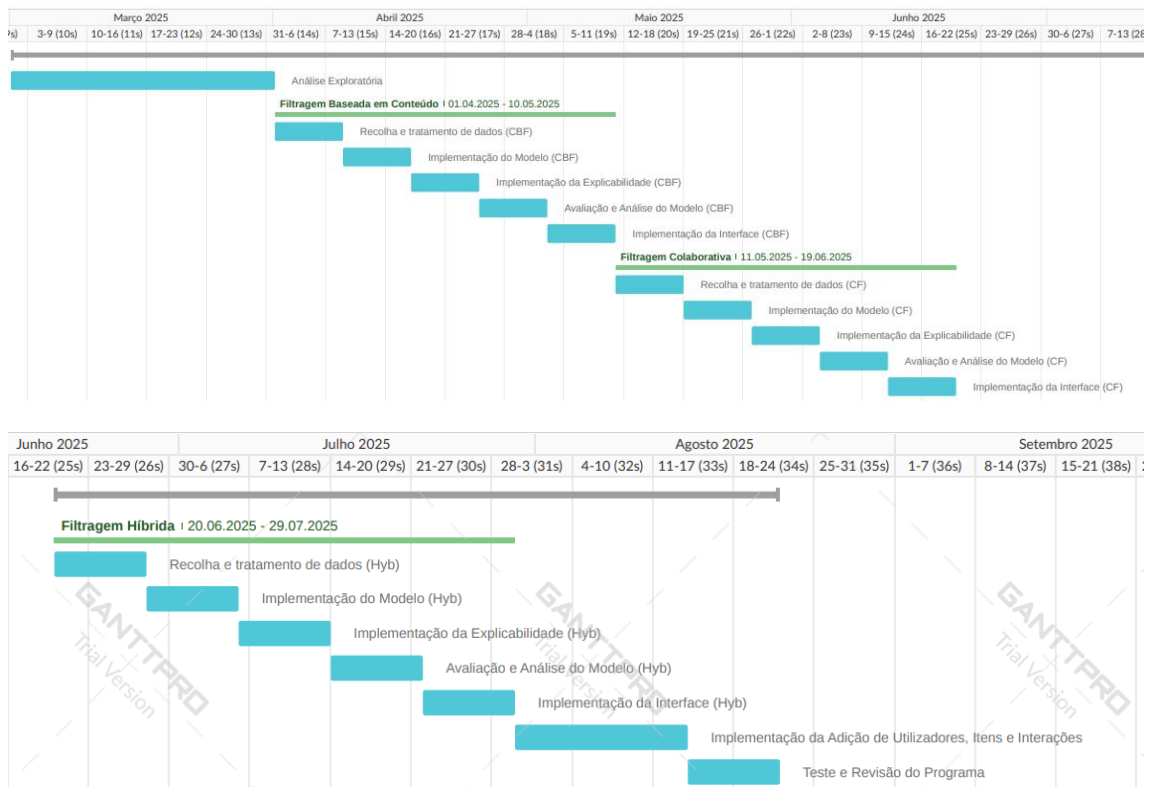


Figura 3 - Diagrama de Gantt

Embora não tenha sido seguida uma metodologia formal como o “*scrum*”, o desenvolvimento do projeto foi orientado por princípios ágeis como iteração contínua, validação incremental dos resultados e adaptação ao longo do processo. Foi definida uma *timeline* geral, organizada por semanas com os objetivos mais amplos e posteriormente os mesmos foram decompostos em tarefas mais simples (*issues*) e atribuídas a dias específicos (*SMART goals*)

5.2 Ferramentas Utilizadas no Desenvolvimento

Durante o desenvolvimento do projeto, foram utilizadas ferramentas amplamente reconhecidas no contexto da engenharia de software:

- **Pycharm (Community Edition v2024.3.5)**: Utilizado como ambiente de desenvolvimento principal, permitindo uma programação eficiente com integração *Git* e execução de scripts *python* diretamente no editor;
- **Git (v 2.45.1.windows.1) e GitHub**: Utilizado para controlo de versões, permitindo guardar o histórico de alterações, gerir diferentes versões do código e facilitar o rastreamento de erros.

Além disso, foi utilizado um ambiente virtual *python* para isolar as dependências do projeto, garantindo consistência entre diferentes execuções e evitando conflitos com outras bibliotecas instaladas no sistema.

Para suporte técnico ao desenvolvimento do sistema, destaca-se o uso das seguintes bibliotecas:

- *PyTorch* (v 2.8.0+cu126) e *Skorch* (v 1.1.0): Para implementação e treino de redes neurais.
- *Scikit-learn* (v1.6.1): Entre outros, para o calculo de curvas de aprendizagem, validação cruzada, *grid search*, e para implementação do modelo *Random Forest*.
- *XGBoost* (v3.0.2): Para implementação de modelos de *extreme gradient boosting*.
- *Pandas* (v2.2.3) e *NumPy* (v2.2.4): Para manipulação de *dataframes* e operações numéricas.
- *SHAP* (v0.47.2): Para a explicabilidade dos modelos, permitindo entender e interpretar as previsões/recomendações dos algoritmos de recomendação.
- *Streamlit* (v 1.46.1): Para construção da interface gráfica interativa que permite a visualização das recomendações e explicações.

De notar ainda que para o desenvolvimento e teste da solução foi utilizado um portátil *OMEN by HP Laptop 15-dc0xxx*, com um processador *Intel Core i7-8750H @ 2.21 GHz*, 16 GB de memória RAM, uma placa gráfica *NVIDIA GeForce GTX 1050*, integrada com a *Intel UHD Graphics 630*, e 250 GB de armazenamento SSD. O sistema operativo utilizado foi o *Windows 11 Home 64-bit*.

6. Desenvolvimento da solução

Esta secção descreve de forma detalhada a implementação prática do sistema de recomendação desenvolvido.

6.1 Arquitetura Geral do Sistema

Nesta secção será apresentada a arquitetura geral do sistema. Diferente da arquitetura conceptual, aqui, será feita uma descrição mais técnica e detalhada (médio nível) dos diferentes componentes/módulos que compõe a solução e como eles interagem entre si para resolver o problema proposto.

- Módulo de Dados: É responsável pela leitura e tratamento/transformação de dados. Consome ficheiros *.csv* e retorna *dataframes* da biblioteca *pandas*;
- Módulo de Modelos: Consome os *dataframes* produzidos pelo módulo de dados e com eles treina modelos de recomendação, retornando objetos do tipo *NeuralNetRegressor*, *RandomForestRegressor* e *XGBRegressor*, dependendo do tipo de filtragem selecionada;
- Módulo de Avaliação: Consome os modelos de recomendação treinados e avalia o seu desempenho utilizando as funções *learning_curve* e *cross_validate* da biblioteca *Scikit-learn*, retornando em ambos os casos um dicionário com os resultados;
- Módulo de Previsão: Consome os modelos de recomendação treinados, bem como os *dataframes* relevantes (*dataframes* com itens não avaliados). Posteriormente, para cada item não avaliado, usando os modelos, o seu rating é previsto e armazenado na coluna "*PredictedRating*" presente depois no *dataframe* retornado;
- Módulo de Explicabilidade: Consome um *dataframe* com itens não avaliados e os respetivos *ratings* previstos (as recomendações a explicar), um *dataframe* com uma amostra de itens avaliados para servir de referência e uma instância do modelo usado nas previsões. Posteriormente, retorna, dependendo do tipo de filtragem em questão, um *ndarray* com os

shap values (o valor associado ao impacto que cada *feature* do item teve na recomendação do mesmo (calculados usando “*explainers*” da biblioteca *shap*)) e/ou um conjunto de explicações textuais e valores de similaridade (no caso de *neighbour based explanation* (calculados usando a função *cosine_similarity* da biblioteca *Scikit-learn*));

- Interface Gráfica: Consome as recomendações, explicações e dados relativos à avaliação de modelos produzidos nos módulos anteriores e , usando a biblioteca *Streamlit* , apresenta estes para o utilizador.

Na imagem 4 é ilustrada a arquitetura geral do sistema descrita.

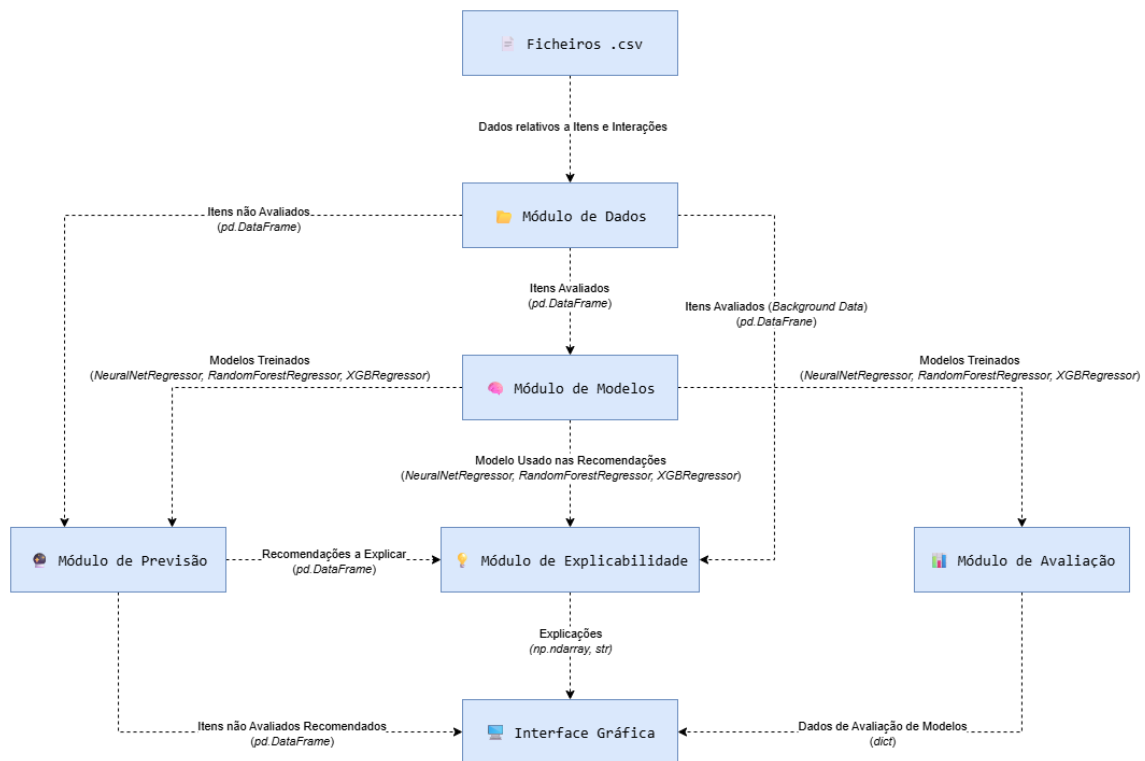


Figura 4- Arquitetura Geral do Sistema

6.2 Preparação e Processamento de Dados

Este módulo (módulo de dados) encontra-se implementado no *core/data_loader.py* e divide-se em alguns passos.

6.2.1 Leitura e Estruturação Inicial

Os dados são obtidos a partir dos ficheiros *movie.csv*, *rating.csv* e *link.csv* do *dataset MovieLens*. A leitura é feita com *pandas*, e os dados são estruturados em *DataFrames*. Os identificadores *userId*, e *movieId* são utilizados como chaves principais para junções e filtragens (como ilustrado na figura 5).

Interações (rating.csv)	Itens (movie.csv)
<u>userId</u>	<u>movieId</u>
<u>movieId</u>	[features]*
rating	

Figura 5 - Estrutura dos dados nos ficheiros do MovieLens

6.2.2 Filtragem e Amostragem de Dados

- Para *Content-Based Filtering* (CBF): As interações presentes no ficheiro *rating.csv* são filtradas para incluir apenas as relativas a um utilizador específico (o selecionado). Posteriormente é realizado um *left join* entre os itens presentes no ficheiro *movie.csv* e as interações filtradas (resultando em um *dataframe* com todos os itens (e *features* associadas) bem como as interações do *user* selecionadas associadas a cada item (como ilustrado na figura 6)).

Filtragem e Amostragem de Dados para CBF
<u>movieId</u>
[features]*
userId
rating

Figura 6 - Filtragem e Amostragem dos dados para CBF

- Para *Collaborative Filtering* (CF): Partindo das interações presentes no ficheiro *rating.csv* e os itens presentes no ficheiro *movie.csv*, é realizado um *join* entre os 2 e posteriormente selecionado um subconjunto de utilizadores (incluindo o utilizador selecionado) e as interações dos mesmos com os itens (como ilustrado na figura 7)(a união é feita para garantir que não são selecionadas interações que referem itens inexistentes).

Filtragem e Amostragem de Dados para CF
<u>userId</u>
<u>movieId</u>
rating

Figura 7 - Filtragem e Amostragem dos dados para CF

6.2.3 Pré-processamento de Géneros (CBF)

No caso da filtragem baseada em conteúdo, o campo “*genres*”, encontra-se originalmente com os géneros todos juntos numa só *string*, apenas separados por “|”. Neste passo efetuamos algumas transformações:

- Divisão da *string* em listas de géneros por filme.
- Aplicação de *one-hot encoding* com *pd.get_dummies*.
- Agrupamento por *movieId* e fusão com o *DataFrame* original.
- Remoção de colunas agora desnecessárias como “*genres*” e “*genre_list*”.

Este processo resulta numa representação binária dos géneros, compatível com cálculos de similaridade e treino de modelos (como ilustrado na figura 8).



Figura 8 - Pré-processamento dos Géneros para CBF

6.2.4 Divisão em Conjuntos de Dados

Após o pré-processamento dos géneros, os itens com interações válidas (*rating* não nulo) são considerados itens avaliados (*rated*) e os restantes são considerados não avaliados (*unrated*).

No caso de *collaborative filtering*, os itens são considerados avaliados quando possuem uma interação associada (independentemente de essa interação pertencer ao *user* selecionado ou não). Já os itens não avaliados são os itens não avaliados pelo utilizador selecionado, mas avaliados por outros utilizadores que não o selecionado (todos os itens presentes no *dataframe unrated*, estão presentes no *rated*, mas não vice-versa)(como ilustrado na figura 9).

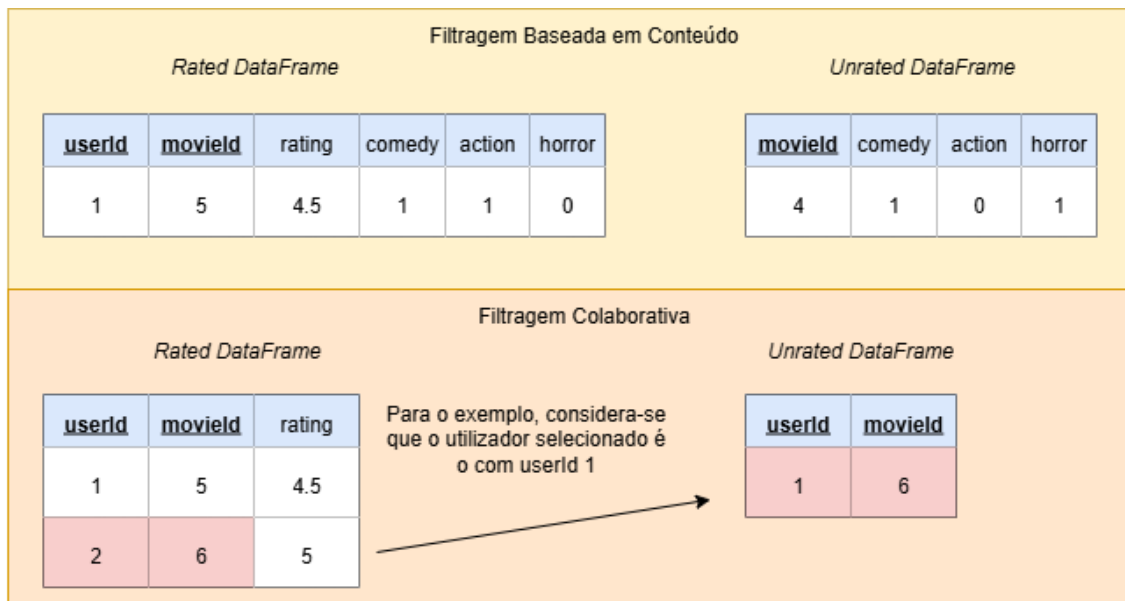


Figura 9 - Divisão dos dados em dataframes *rated* e *unrated*

6.2.5 Limpeza e Indexação

Dependendo do tipo de filtragem, as colunas desnecessárias são removidas (ex: "*timestamp*", "*title*", "*tmdbId*") e os *dataframes* são indexados por *movieId* (CBF) ou mantidos com identificadores explícitos (*userId*, *movieId*) (CF).

6.2.6 Persistência dos Dados

Sempre que possível, os dados transformados são gravados em ficheiros .csv, evitando repetições de processamento desnecessárias em execuções futuras. Estes ficheiros são automaticamente carregados quando disponíveis.

6.3 Implementação dos Modelos de Recomendação

Este módulo (módulo de modelos) encontra-se implementado no *core/model.py*.

6.3.1 Filtragem Baseada em Conteúdo

Como referido anteriormente, na filtragem baseada em conteúdo, as *features*/conteúdo de um item (variáveis independentes) são usadas para fazer a recomendação, isto é, tentar prever o seu *rating* (variável dependente) ou de forma genérica, quanto interesse o utilizador terá no referido item.

Para efeito de comparação, tanto ao nível de desempenho como de interpretabilidade, são implementados 3 modelos distintos:

- Redes Neurais: Utilizando a biblioteca *PyTorch* (com integração via *Skorch*);
- *Random Forest Regressor*: Utilizado através da biblioteca *Scikit-learn*;
- *XGBoost Regressor (Extreme Gradient Boost)*: Utilizado através da biblioteca *XGBoost*.

A seleção do modelo a utilizar é feita pelo utilizador na própria *user interface* (a rede neuronal é usada por defeito) e a parametrização dos diferentes modelos é feita dinamicamente no ficheiro *configs.py*. A persistência dos modelos treinados é realizada de acordo com o tipo de modelo:

- Redes Neurais: Com a função *save_params* via *Skorch* (ficheiro *.pkl*) e respetivos metadados em *.json*;
- *Random Forest*: Com recurso à biblioteca *joblib* (ficheiro *.pkl*)
- *XGBoost*: Utilizando a função *save_model* da biblioteca *XGBoost* (ficheiro *.json*)

De notar que no caso da CBF, o modelo é treinado com os dados relativos a um perfil de utilizador específico, com o objetivo de o modelo aprender os gostos desse utilizador, por consequência é necessário um modelo por cada perfil de utilizador e a qualidade do modelo é previsivelmente afetada pela quantidade de interações que dado utilizador tenha tido (com mais interações, o modelo tem mais dados para treinar e “aprender”). De notar ainda que, sendo as previsões baseadas nas *features* de um item, o mesmo modelo pode ser utilizado para novos itens desde que estes tenham as suas *features* devidamente descritas, contudo, no caso de interações, novas interações apenas terão efeito na previsão aquando do retreino do modelo.

6.3.2 Filtragem Colaborativa com *Embeddings*

Na filtragem colaborativa são usadas as interações de diferentes utilizadores com dado item para decidir se o item é ou não do interesse do utilizador selecionado. A lógica é que “utilizadores que gostaram dos mesmos itens no passado, provavelmente gostarão dos mesmos itens no futuro”.

Nesta abordagem, cada utilizador e cada item são representados por vetores (*embeddings*) aprendidos automaticamente durante o treino (os vetores, inclusive do item, são derivados das interações de utilizadores com dado item (as *features*/conteúdo do item não são usadas)).

A entrada do modelo consiste num par (*user_id*, *movie_id*), que é posteriormente convertido nos respetivos *embeddings*. Estes vetores são posteriormente concatenados e passados por uma rede neural que prevê a avaliação que dado utilizador dará a um item.

A dimensão dos *embeddings*, o número de camadas ocultas, o número de neurónios por camada entre outros parâmetros, são definidos no ficheiro *configs.py*.

A persistência do modelo treinado segue a mesma lógica das redes neuronais para a filtragem baseada no conteúdo, sendo o modelo guardado num ficheiro *pickle* e os respetivos metadados num ficheiro *json*.

De notar que, tendo a CF sido implementado com *embeddings*, neste caso apenas redes neuronais foram utilizadas. De notar ainda que, tendo cada perfil de utilizador e item um *embedding* associado, o mesmo modelo pode ser utilizado para fazer previsões, relativas a qualquer perfil de utilizado utilizador e qualquer item, desde que estes estejam presentes no *dataset* usado durante o treino, contudo, por consequência do mesmo, novos utilizadores ou itens, exigem o treino de um novo modelo (uma vez que estes não tem *embedding* associados), além disso, novas interações apenas tem efeito na previsão aquando do retreino do modelo (aquando do calculo de novos *embeddings*).

6.3.3 Filtragem Híbrida

Na filtragem híbrida são usadas tanto as interações de diferentes utilizadores com dado item, como as *features*/conteúdo do item para fazer a recomendação.

Nesta abordagem, cada utilizador e cada item são representados por vetores, sendo o vetor do item as *features*/conteúdo do mesmo e o vetor do utilizador o resultado da normalização da média ponderada das *features* dos itens avaliados, com o rating atribuído aos mesmos (como exemplificado na figura 10).

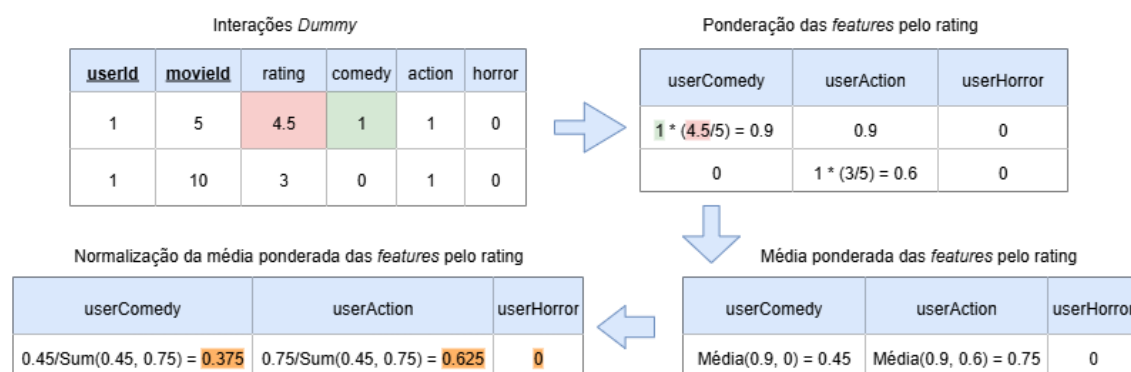


Figura 10 - Cálculo do vetor de utilizador em filtragem híbrida

Assim é utilizado o par de vetores de utilizador e item, concatenados, para passados por uma rede neural, prever a avaliação que o utilizador em questão dará ao referido item.

Tanto a parametrização dos modelos, como a persistência dos mesmos, segue a mesma lógica da filtragem baseada em conteúdo.

De notar que, sendo o vetor de utilizador calculado aquando da seleção do tipo de filtragem e recalculado aquando da tomada de ações que podem modificar este (como a adição de novas interações), o mesmo modelo, contrariamente ao implementado na CF, pode ser usado para previsões com qualquer perfil de utilizador ou item, independentemente de estes estarem ou não presentes no *dataset* usado durante o treino.

6.3.3 Estratégia de Treino e Validação

Independentemente do modelo, como referido anteriormente, os hiperparâmetros do mesmo são definidos dinamicamente no ficheiro *configs.py*.

De forma a garantir uma seleção mais eficaz dos mesmos, foi também implementada a técnica de *Grid Search* com validação cruzada (*GridSearchCV*) da biblioteca *Scikit-learn*. Este processo permite testar diferentes combinações de hiperparâmetros, avaliando o desempenho de cada combinação através de uma métrica de *scoring*, como o RMSE ou R^2 , em múltiplos *folds* de dados e posteriormente selecionar a com melhor resultado.

Os resultados da *Grid Search* são posteriormente guardados em um ficheiro *.csv*.

6.4 Avaliação e Análise de Desempenho dos Modelos

Este módulo (módulo de avaliação) encontra-se implementado no *core/evaluation.py*.

A avaliação do desempenho dos modelos é bastante importante, pois permite comparar abordagens e analisar a capacidade de preditiva e de generalização de cada um. Além disso, ajuda a identificar problemas como *overfitting* (o modelo é demasiado complexo e “decora” os dados de treino (obtendo bons resultados durante o treino, mas fracos resultados durante a validação)) ou *underfitting* (o modelo é demasiado simples e por isso incapaz de detetar os padrões nos dados (obtendo fracos resultados tanto durante o treino como durante a validação)).

Para o efeito, são utilizadas duas técnicas, sendo elas:

- Validação Cruzada: A implementação é feita recorrendo à função *cross_validate* da biblioteca *Scikit-learn*;
- Curvas de Aprendizagem: A implementação é feita, recorrendo à função *learning_curve* da biblioteca *Scikit-learn*.

Em ambas as técnicas, os parâmetros de configuração encontram-se definidos no ficheiro *configs.py* e o sistema tenta carregar os resultados de validação cruzada e curvas de aprendizagem previamente armazenadas. Quando estes não se encontram disponíveis, então novos são calculados (e armazenados).

6.5 Previsão das Recomendações

Este módulo (módulo de previsão) encontra-se implementado no *core/prediction.py*.

O módulo de previsão é responsável por, usando o *dataframe* de “unrated_items” produzido em “Preparação e Processamento de Dados”, juntamente com os modelos desenvolvidos em “Implementação de Modelos de Recomendação”, gerar “*PredictedRatings*” para esses itens (tendo em conta o utilizador selecionado). Posteriormente, o *dataframe* é ordenado de forma decrescente de “*PredictedRating*” e dessa forma os itens são selecionados para serem recomendados.

De notar que o módulo é compatível com todos os tipos de modelos e filtragens implementados e que processa as inferências em *batch*, reduzindo significativamente o *overhead* que múltiplas chamadas individuais teriam.

6.6 Explicabilidade das Recomendações

Este módulo (módulo de explicabilidade) encontra-se implementado no *core/explainability.py*.

O módulo de explicabilidade tem como objetivo explicar e justificar o porquê de dado item ser recomendado ao utilizador selecionado. Os modelos utilizados (redes neuronais, *random forest* e *xgboost*) para fazer as recomendações são modelos de “caixa preta”, isto é, o seu funcionamento interno é ou opaco, ou de tal modo complexo, que não é facilmente interpretável pelo ser humano,

assim é importante o uso destas explicações para mais facilmente haver confiança e credibilidade nas previsões do modelo.

Para o efeito, foram implementadas 2 estratégias, as explicações baseadas em vizinhos (para a filtragem colaborativa) e as explicações SHAP (para a filtragem baseada em conteúdo) (no caso da filtragem híbrida, ambas as explicações são usadas).

6.6.1 Explicações Baseadas em Vizinhos

Para a filtragem colaborativa (ou híbrida), as explicações são baseadas em vizinhos, isto é, são construídas com base em utilizadores “semelhantes” ao selecionado.

A lógica implementada segue os seguintes passos:

1. Identificação de utilizadores relevantes: O sistema identifica os utilizadores que avaliaram o item recomendado;
2. Extração dos vetores de utilizador: O sistema identifica os vetores de utilizador dos utilizadores marcados como relevantes (presentes no *dataframe* “*rated_items*”);
3. Cálculo da similaridade: É calculada a similaridade ponderada ($\text{similaridade} \times \text{rating atribuído}$) entre o vetor do utilizador selecionado e os vetores dos utilizadores que avaliaram o item (é utilizada a função *cosine_similarity* da biblioteca *Scikit-learn*, para o propósito).
4. Seleção dos top-k mais semelhantes: São selecionados os utilizadores com maior similaridade ao utilizador selecionado.
5. Geração da explicação: Para cada um destes utilizadores, o valor de similaridade ao utilizador selecionado e a classificação que atribuíram ao item recomendado, são agregadas numa explicação textual, como exemplificado na figura 11.

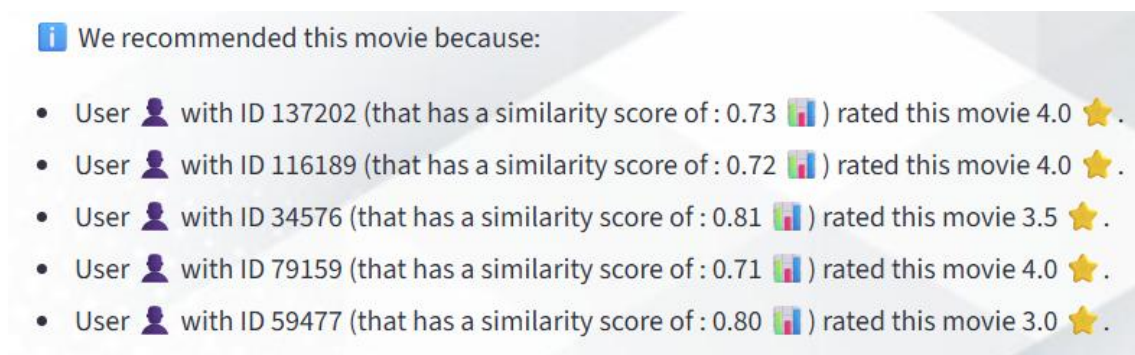


Figura 11 - Explicações baseadas em vizinhos

6.6.2 Explicações com SHAP

Para a filtragem baseada em conteúdo (ou híbrida), usamos a técnica pós-hoc SHAP (SHapley Additive exPlanations), isto é, as explicações são baseadas na importância que cada *feature* (variável independente) tem para a previsão do rating (variável dependente).

A lógica implementada segue os seguintes passos:

1. Preparação de dados de fundo: Um subconjunto aleatório de filmes avaliados é utilizado como conjunto de referência para as explicações SHAP (*background_data*).
2. Seleção do “explicador” a usar: Dependendo do modelo utilizado, aplica-se:
 - *DeepExplainer* para redes neurais;
 - *TreeExplainer* para *Random Forest* e *XGBoost*;
 - *KernelExplainer* como abordagem agnóstica (está implementado, mas não é usado, uma vez que os outros *explainers* já cobrem os modelos usados).
3. Geração dos valores SHAP: Estes representam a contribuição individual de cada *feature* (variável independente) para a avaliação prevista (variável dependente).
4. Filtragem e visualização: *Features* com valores pouco significativos são excluídas e as restantes são ilustradas em um gráfico de barras e um gráfico circular (como ilustrados na figura 12 e 13).

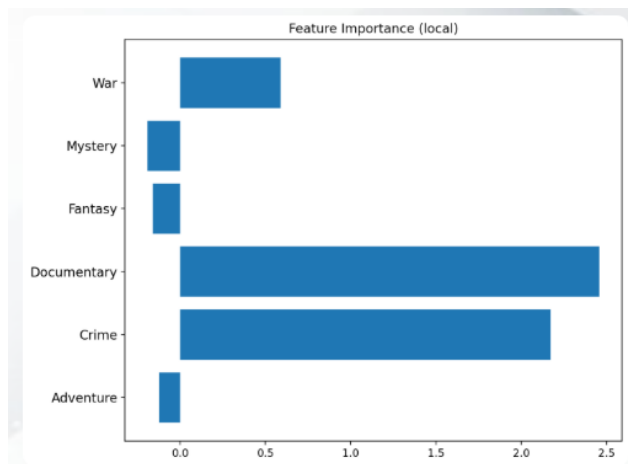


Figura 13- Gráfico de barras de valores SHAP

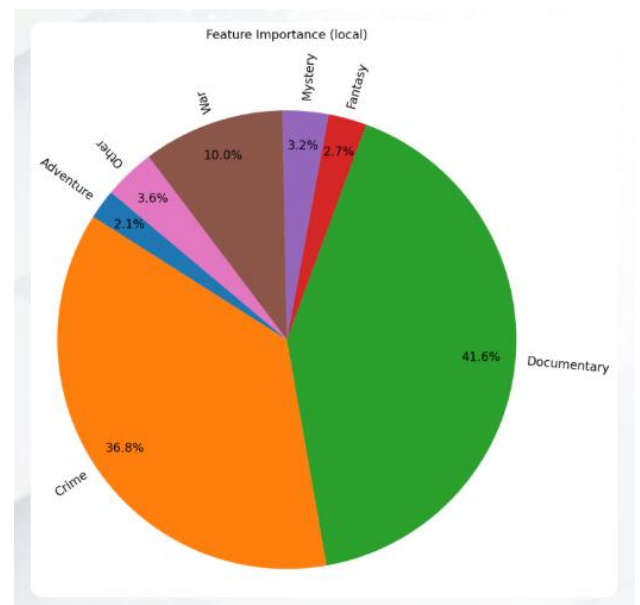


Figura 12- Gráfico circular de valores SHAP

De notar que tanto a “presença” ou “ausência” de uma *feature* em um item, contribui para a previsão do seu rating (uma *feature* continua a ser uma variável independente, mesmo que o seu valor no vetor do item em questão seja 0).

6.7 Interface Interativa com *Streamlit*

6.7.1 Componentes e Funcionalidades Visuais

A interface de utilizador encontra-se implementada principalmente no *ui/app.py*, mas faz uso de vários outros ficheiros auxiliares presentes no diretório *ui/* que por sua vez interagem com ficheiros do diretório *core/*.

A interface de utilizador é implementada com a biblioteca *Streamlit* e é responsável por apresentar e permitir ao utilizador interagir com as restantes funcionalidades do sistema.

De entre os usos, destacam-se:

Permite ver, filtrar e navegar entre os itens recomendados ao utilizador selecionado, como demonstrado na figura 14.

De notar que também permite fazer o mesmo com os itens avaliados pelo perfil de utilizador selecionado.

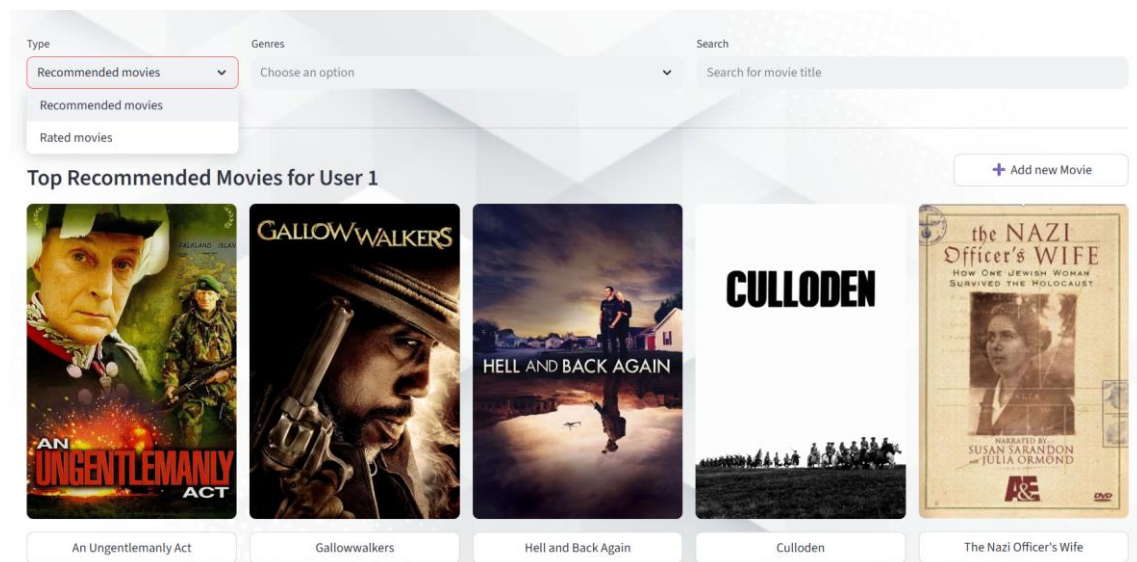


Figura 14- Listagem dos itens recomendados

Permite selecionar e ver os detalhes de um filme recomendado, como demonstrado na figura 15. De notar que permite fazer o mesmo com os filmes avaliados pelo utilizador selecionado.

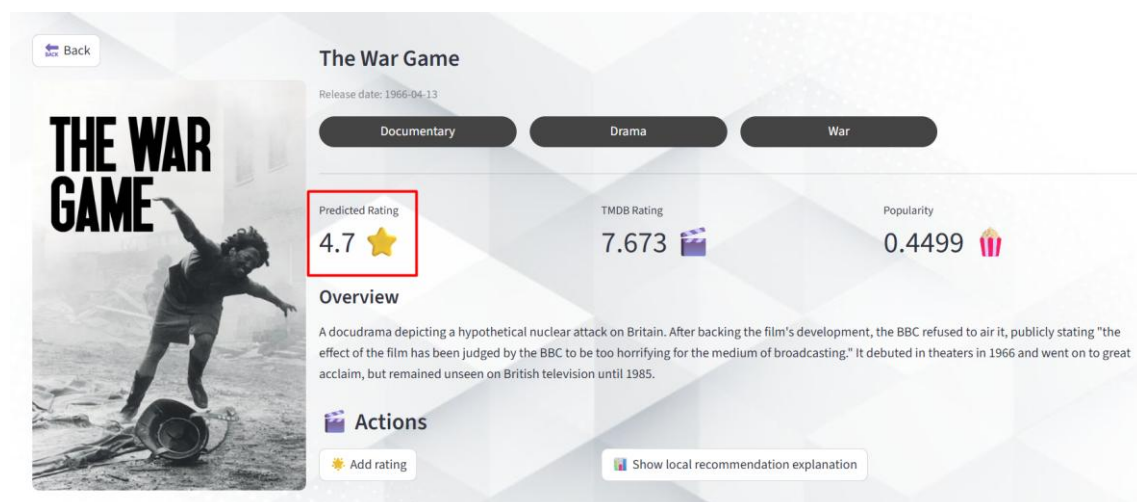


Figura 15 - Detalhes de um item recomendado

No caso de um item ser recomendado, permite ver as explicações locais associadas a essa recomendação, como demonstrado nas figuras 16 e 17.

💡 Neighbour Based Explanation (Local)

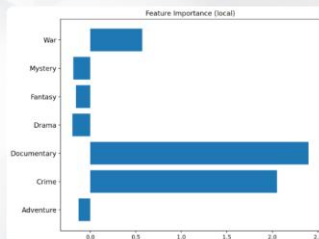
i We recommended this movie because:

- User  with ID 137202 (that has a similarity score of : 0.73 ) rated this movie 4.0 .
- User  with ID 116189 (that has a similarity score of : 0.72 ) rated this movie 4.0 .
- User  with ID 34576 (that has a similarity score of : 0.81 ) rated this movie 3.5 .
- User  with ID 79159 (that has a similarity score of : 0.71 ) rated this movie 4.0 .
- User  with ID 59477 (that has a similarity score of : 0.80 ) rated this movie 3.0 .

Figura 16 – Explicações locais baseadas em vizinhos

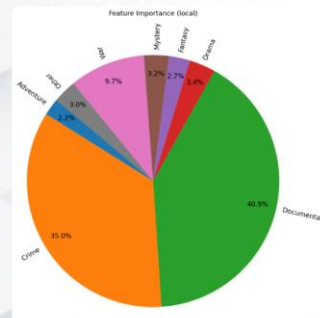
💡 SHAP Explanation (Local)

Dep



i Bar Chart Explanation:

- Each bar represents a feature (e.g., movie genre, presence of an actor, etc.).
- The height of the bar shows how much that feature contributed to the model's prediction.
- Positive values indicate the feature increased the predicted rating, while negative values indicate it decreased the predicted rating.
- Even if a movie does not have a certain feature (for example, it's not a specific genre), the bar can still have a non-zero value. This happens because the model considers the impact of the absence of the feature on the prediction compared to the average.



i Pie Chart Explanation:

- Each slice shows the relative contribution of each feature to the model's prediction.
- The size of the slice indicates how much that feature influenced the prediction relative to the total.
- Even features that the movie does not have can appear as slices because the model accounts for the effect of both presence and absence.
- Very small slices can be grouped into "Other" to simplify visualization.
- Unlike the bar chart, the pie chart shows magnitude only, not effect direction.

Figura 17 - Explicações locais SHAP

Permite seleccionar diferentes perfis de utilizador, como demonstrado na figura 18, bem como ver as explicações *SHAP* globais associadas ao perfil de utilizador seleccionado, como demonstrado na figura 19.



User

Selected user id: 1

User id: 1 number of rated movies: 185

User id: 1 number of rated movies: 185

User id: 2 number of rated movies: 61

User id: 3 number of rated movies: 187

User id: 4 number of rated movies: 28

User id: 5 number of rated movies: 66

Figura 18 - Seleção de perfil de utilizador



Figura 19 - Explicações globais SHAP

Permite seleccionar diferentes tipos de filtragem e modelos de recomendação, como demonstrado na figura 20, bem como ver a avaliação (*learning curve* e *cross validation*) associada ao modelo seleccionado, como demonstrado na figura 21 e 22.

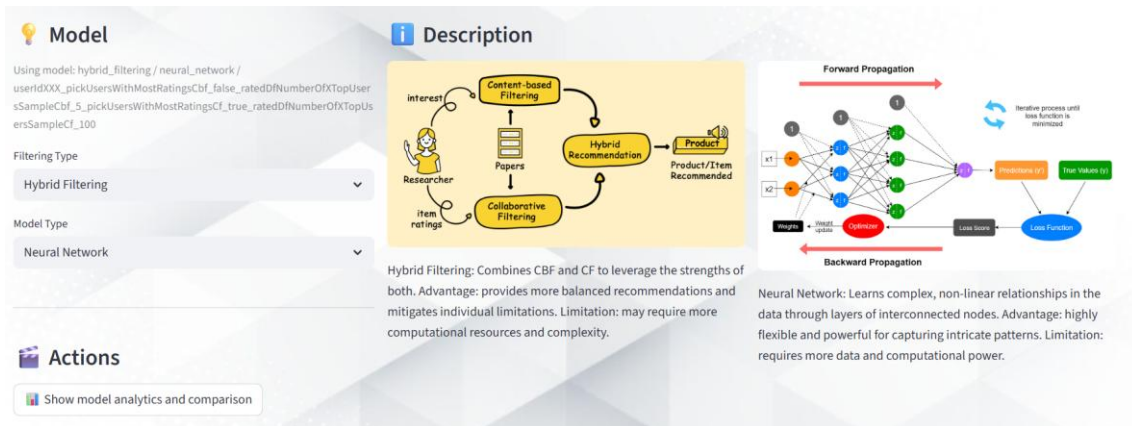


Figura 20 - Seleção de modelo e tipo de filtragem



Figura 21 - Análise e avaliação do modelo selecionado (learning curve)

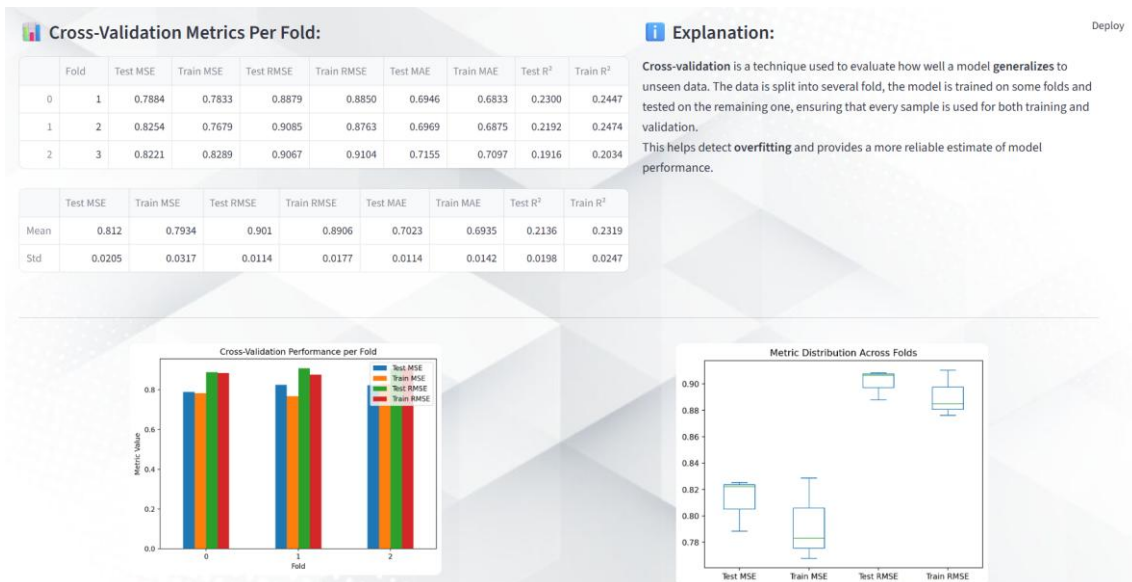


Figura 22- Análise e avaliação do modelo selecionado (cross validation)

Por fim, dependendo do tipo de filtragem selecionado, permite adicionar novos perfis de utilizador, itens e interações, como demonstrado na figura 23, 24 e 25.



User

Selected user id: 1

User id: 1 number of rated movies: 185

▼



Actions

 Create new User



Show global recommendation explanation



Ratings

User 1 has rated 185 unique movies.

	title
12791	Ghosts of Cité Soleil (2006)
12702	little cat
12796	Rain Over Santiago (Il pleut sur Santiago) (1976)
12793	Iria: Zeiram the Animation (1993)
12794	Dikkenek (2006)
12803	Berserk: The Golden Age Arc - The Egg of the King (2012)
12788	Dark City (1998)
12797	First Squad: The Moment of Truth (2009)
12800	Galapagos Affair: Satan Came to Eden, The (2013)
5325	Star Kid (1997)

Figura 23 - Adição de novo perfil de utilizador



BRANCA DE NEVE

Title

Branca de Neve

Genres

Adventure x Animation x

Release date

2025/09/07

Poster's Url

https://novizer.com/content/movies/posterPt/342/41369.webp

TMDB Rating

5,0

- +

Popularity

5,0

- +

Overview

When the Evil Queen orders her stepdaughter killed out of envy of her beauty, Snow White flees the kingdom. She wanders through the forest and finds a small house where seven friendly dwarves live, with whom she begins to live. However, the girl is not safe yet, as the witch plans to end her life with a poisoned apple.

Actions

 Add Movie

 Cancel

Figura 24 - Adição de novo item

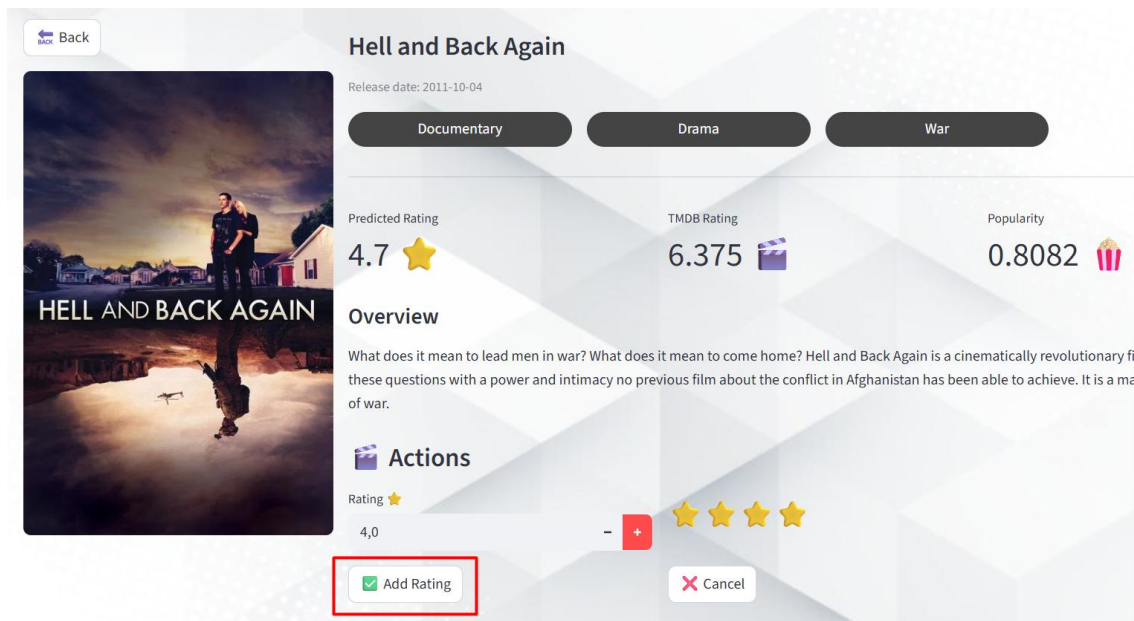


Figura 25 - Adição de nova interação

6.7.2 Integração com API Externa para Enriquecimento dos Itens Apresentados

Para uma apresentação mais rica e apelativa dos itens, o sistema busca obter informações adicionais sobre os itens a serem apresentados, como títulos detalhados, sinopses, *posters* e outros dados (não presentes no *dataset* do *MovieLens*), a partir da *API* do *The Movie Database (TMDb)*.

Para isso, os seguintes passos são seguidos:

- **Requisições *HTTP*:** Para cada item recomendado com um identificador *TMDb* válido, é feita uma chamada *HTTP* à *API* do *TMDb* usando a biblioteca *requests*. Os parâmetros incluem a chave da *API* e o idioma desejado.
- **Concorrência para performance:** Para acelerar o processo, as chamadas são feitas em paralelo utilizando o *ThreadPoolExecutor* do módulo *concurrent.futures* (faz parte da biblioteca do *Python*).
- **Tratamento e junção dos dados:** As respostas são processadas para extrair as informações relevantes (título, *overview*, *URL* do *poster*, data de lançamento, entre outros). Estes dados são convertidos em um *DataFrame* e posteriormente juntados com o *DataFrame* de itens recomendados.

6.8 Geração de *Logs*

Para monitorizar o funcionamento da aplicação e facilitar a deteção e resolução de erros e *bottlenecks* durante o desenvolvimento e utilização da aplicação, foi implementado um sistema de *logs*.

Nesse sentido, foi implementada a função *write_log*, que, recebendo uma mensagem de texto, a grava no ficheiro *logs.txt*.

De notar que é adicionando um *timestamp* para registar o momento exato do evento, a escrita é efetuada imediatamente com *flush()* e opcionalmente a mensagem pode também ser demonstrada na consola.

CAP IV – Discussão dos resultados

7. Apresentação e discussão dos resultados

Uma vez que a exposição e descrição das diferentes funcionalidades implementadas já foi feita no capítulo anterior, nesta secção o foco estará em apresentar uma análise do desempenho de diferentes modelos de *machine learning* desenvolvidos para fazer as recomendações.

Como mencionado, foram implementados 3 tipos de filtragem, baseada em conteúdo, colaborativa e híbrida. Para para a CBF e híbrida foram implementados modelos do tipo rede neuronal, *random forest* e *xgboost*. Já para CF, tendo sido usados *embeddings*, foi apenas usada a rede neuronal.

Para efeitos de análise, foi utilizado o mesmo perfil de utilizador com id 118205, que possui 9254 interações associadas.

Na tabela 13 são indicados os hiperparâmetros de configuração usados na rede neuronal para CBF.

Rede Neuronal (CBF)

Parâmetro	Valor
hidden_dim	[128, 128]
learning_rate	0.1
num_epochs	70
batch_size	64
iterator_train_shuffle	True
dropout	0

Tabela 12 - Parâmetros de treino de rede neuronal (cbf)

Na tabela 14 são indicados os hiperparâmetros de configuração usados na rede neuronal para CF.

Rede Neuronal (CF)

Parâmetro	Valor
hidden_dim	[32, 32]
learning_rate	0.0005
num_epochs	70
batch_size	256
iterator_train_shuffle	True
embedding_dim	64
dropout	0.5

Tabela 13 - Parâmetros de treino de rede neuronal (cf)

Na tabela 15 são indicados os hiperparâmetros de configuração usados na rede neuronal para filtragem híbrida.

Rede Neuronal (Filtragem Híbrida)

Parâmetro	Valor
hidden_dim	[128, 128]
learning_rate	0.01
num_epochs	70
batch_size	256
iterator_train_shuffle	True
dropout	0

Tabela 14 - Parâmetros de treino de rede neuronal (híbrida)

Na tabela 16 são indicados os hiperparâmetros de configuração usados na *random forest* para CBF.

Random Forest (CBF)

Parâmetro	Valor
n_estimators	100
max_depth	10
random_state	42

Tabela 15 - Parâmetros treino de random forest (cbf)

Na tabela 17 são indicados os hiperparâmetros de configuração usados na *random forest* para filtragem híbrida.

Random Forest (Filtragem Híbrida)

Parâmetro	Valor
n_estimators	100
max_depth	10
random_state	42

Tabela 16 - Parâmetros treino de random forest (híbrida)

Na tabela 18 são indicados os hiperparâmetros de configuração usados no *xgboost* para CBF.

Extreme Gradient Boost (CBF)

Parâmetro	Valor
n_estimators	100
max_depth	10
random_state	42
learning_rate	0.1
objective	reg:squarederror

Tabela 17 - Parâmetros treino de xgboost (cbf)

Na tabela 19 são indicados os hiperparâmetros de configuração usados no *xgboost* para filtragem híbrida.

Extreme Gradient Boost (Filtragem Híbrida)

Parâmetro	Valor
n_estimators	100
max_depth	10
random_state	42
learning_rate	0.1
objective	reg:squarederror

Tabela 18 - Parâmetros treino de xgboost (híbrida)

De notar que em filtragem colaborativa e filtragem híbrida, foi utilizado um *subset* dos utilizadores, correspondendo aos 100 perfis com mais interações (e o utilizador selecionado), resultando em um *dataset* com 346618 interações sobre as quais os modelos foram treinados.

Nas Figuras 26 e 27 são apresentadas as médias das métricas R^2 , MAE, RMSE e MSE dos modelos, obtidas nos conjuntos de treino e validação através de validação cruzada com 3 *folds*.

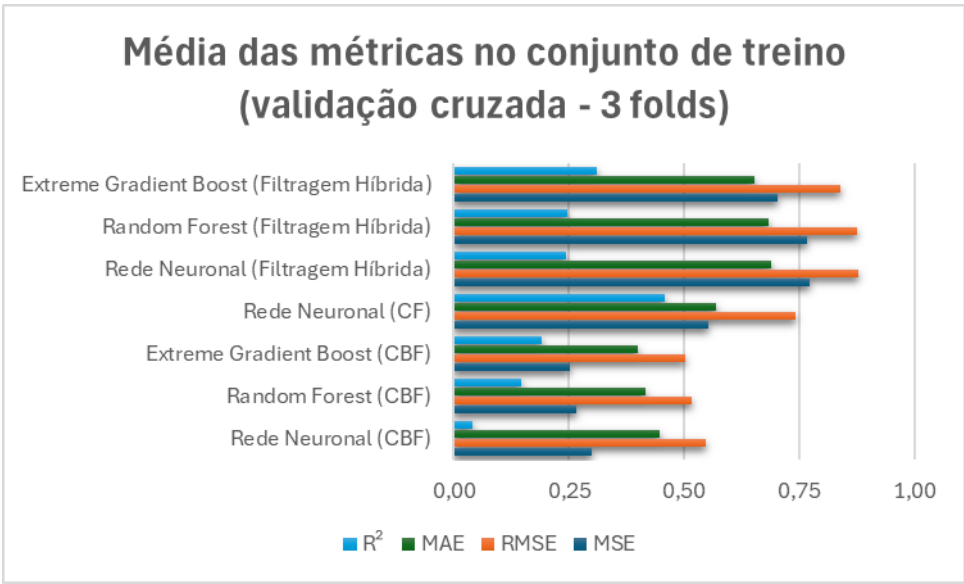


Figura 26 - Média das métricas no conjunto de treino (validação cruzada com 3 folds)

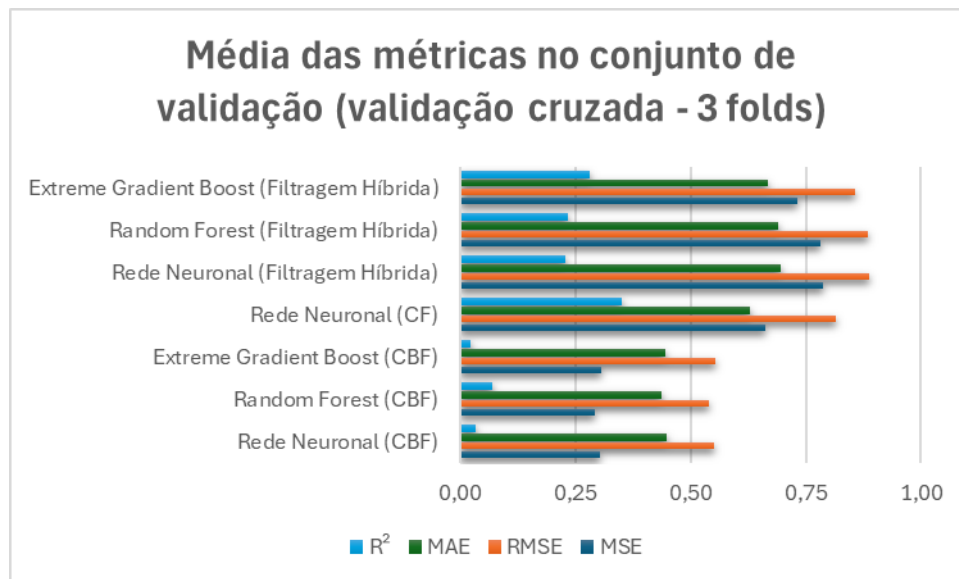


Figura 27 - Média das métricas no conjunto de validação (validação cruzada com 3 folds)

Da observação dos gráficos apresentados, algo que podemos notar é que, independentemente do tipo de modelo utilizado (rede neuronal, *random forest* ou *xgboost*), os modelos de filtragem híbrida ou *CF*, treinados com um *dataset* maior (um *dataset* que inclui interações de vários perfis de utilizador), tendem a apresentar um R^2 superior aos de *CBF*, treinados com um *dataset* menor (com as interações relativas apenas ao perfil de utilizador selecionado), isto é, o modelo consegue por norma capturar melhor a variância nos dados.

O modelo de *CF* que usa a rede neuronal com *embeddings*, apresentou um R^2 superior e métricas de erro menor aos de filtragem híbrida.

Algo também a notar é que apesar de os modelos de *CBF* possuírem por norma um R^2 inferior aos modelos de filtragem híbrida e *CF*, por norma, possuem também um menor erro (como observado com as métricas MAE, RMSE e MSE), o que indica previsões mais conservadoras, isto é, próximas da média global.

No que diz respeito à diferenciação entre diferentes modelos, apesar do custo computacional substancialmente superior envolvido no treino de redes neuronais, os 3 tipos de modelos implementados, obtiveram resultados bastante semelhantes.

8. Apresentação e discussão dos impedimentos e/ou constrangimentos

O principal desafio ao longo do projeto consistiu em organizar e estruturar o sistema de modo a permitir a sua expansão e evolução de forma consistente e coerente.

Este desafio surgiu pelo facto de se ter optado por usar *python* e um paradigma procedural, em detrimento de outras linguagens e paradigmas como java e programação orientada a objetos, as quais foram utilizadas mais frequentemente durante o percurso académico.

Do ponto de vista mais técnico, ao longo do projeto, foram utilizadas diferentes bibliotecas que esperam dados em formatos diferentes, necessitando por isso da implementação de todo um “*middleware*” para garantir a compatibilidade e correto funcionamento do sistema.

Também foi dada bastante atenção ao tempo que as diferentes tarefas necessitam para ser executadas, tendo sido implementadas uma série de soluções, desde a de gravar dados localmente e em memória, a de implementar paralelismo em alguns processos mais morosos, a de processar elementos em *batch*, entre outros.

CAP V – Conclusão

9. Reflexão crítica dos resultados

Como demonstrado nos capítulos anteriores, o sistema implementado cumpre com sucesso todos os requisitos, alcançando assim os objetivos propostos.

Em relação aos modelos treinados, as métricas de avaliação são modestas, em parte, refletindo as diferenças e limitações entre os diferentes tipos de filtragem (como descrito na fundamentação teórica).

10. Conclusão e trabalho futuro

Apesar dos muitos desafios, que surgiram ao longo do desenvolvimento do sistema, com persistência, de uma forma metódica e sistemática, fui capaz de os solucionar.

Naturalmente, existem sempre melhorias, existem sempre outras formas de fazer mais e melhor e o simples facto de termos consciência disso, demonstra a nossa contínua evolução e crescimento.

Dito isto, estou bastante satisfeito com o trabalho desenvolvido.

Para trabalhos futuros, seria interessante explorar novas técnicas de recomendação, otimizações de performance e integração de métodos híbridos adicionais, de forma a melhorar ainda mais a eficácia do sistema.

11. Referências

- [1] D. Roy and M. Dutta, "A systematic review and research perspective on recommender systems," *J Big Data*, vol. 9, no. 1, Dec. 2022, doi: 10.1186/s40537-022-00592-5.
- [2] R. Burke, "Recommender Systems: An Overview," in *Encyclopedia of Machine Learning*, C. Sammut and G. I. Webb, Eds. Boston, MA: Springer, 2011, pp. 829–838. doi: 10.1007/978-0-387-30164-8_652.
- [3] L. Lü, M. Medo, C. H. Yeung, Y.-C. Zhang, Z.-K. Zhang, and T. Zhou, "Recommender Systems," Feb. 2012, doi: 10.1016/j.physrep.2012.02.006.
- [4] E. Çano and M. Morisio, "Hybrid recommender systems: A systematic literature review," 2017, *IOS Press*. doi: 10.3233/IDA-163209.
- [5] R. Chen, Q. Hua, Y. S. Chang, B. Wang, L. Zhang, and X. Kong, "A survey of collaborative filtering-based recommender systems: from traditional methods to hybrid methods based on social networks," *IEEE Access*, vol. 6, pp. 64301–64320, 2018, doi: 10.1109/ACCESS.2018.2877208.
- [6] S. Sharma, V. Rana, and M. Malhotra, "Automatic recommendation system based on hybrid filtering algorithm," *Educ Inf Technol (Dordr)*, vol. 27, no. 2, pp. 1523–1538, Mar. 2022, doi: 10.1007/s10639-021-10643-8.
- [7] H. Truong Thanh Nguyen, H. Cao, K. Vo Thanh Nguyen, and N. Dinh Khoi Pham, "Evaluation of Explainable Artificial Intelligence: SHAP, LIME, and CAM Evaluation of Explainable Artificial Intelligence: SHAP, LIME, and CAM Hung Quoc Cao," 2021. [Online]. Available: <https://www.researchgate.net/publication/362165633>
- [8] A. Vultureanu-Albisi and C. Badica, "Recommender systems: An explainable AI perspective," in *2021 International Conference on INnovations in Intelligent SysTems and Applications, INISTA 2021 – Proceedings*, Institute of Electrical and Electronics Engineers Inc., Aug. 2021. doi: 10.1109/INISTA52262.2021.9548125.
- [9] D. Afchar, A. B. Melchiorre, M. Schedl, R. Hennequin, E. V. Epure, and M. Moussallam, "Explainability in music recommender systems," *AI Mag*, vol. 43, no. 2, pp. 190–208, Jun. 2022, doi: 10.1002/aaai.12056.
- [10] M. Ravi, A. Negi, and S. Chitnis, "A Comparative Review of Expert Systems, Recommender Systems, and Explainable AI," in *2022 IEEE 7th International conference for Convergence in Technology, I2CT 2022*, Institute of Electrical and Electronics Engineers Inc., 2022. doi: 10.1109/I2CT54291.2022.9824265.
- [11] P. Nagaraj, V. Muneeswaran, A. Dharanidharan, K. Balanathanan, M. Arunkumar, and C. Rajkumar, "A Prediction and Recommendation System for Diabetes Mellitus using XAI-based Lime Explainer," in *International Conference on Sustainable Computing*

and Data Communication Systems, *ICSCDS 2022 – Proceedings*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 1472–1478. doi: 10.1109/ICSCDS53736.2022.9760847.

- [12] A. Vultureanu-Albiși and C. Bădică, "A survey on effects of adding explanations to recommender systems," in *Concurrency and Computation: Practice and Experience*, John Wiley and Sons Ltd, Sep. 2022. doi: 10.1002/cpe.6834.
- [13] C. V. Roberts, E. Elahi, and A. Chandrashekar, "On the Bias–Variance Characteristics of LIME and SHAP in High Sparsity Movie Recommendation Explanation Tasks," Jun. 2022, pp. 1–9. doi: 10.48550/arXiv.2206.04784.
- [14] G. Schwalbe and B. Finzel, "A comprehensive taxonomy for explainable artificial intelligence: a systematic survey of surveys on methods and concepts," *Data Min Knowl Discov*, vol. 38, no. 5, pp. 3043–3101, Sep. 2024, doi: 10.1007/s10618-022-00867-8.
- [15] R. Widayanti, M. Heru, R. Chakim, C. Lukita, U. Rahardja, and N. Lutfiani, "Improving Recommender Systems using Hybrid Techniques of Collaborative Filtering and Content-Based Filtering," *Journal of Applied Data Sciences*, vol. 4, no. 3, pp. 289–302, 2023.
- [16] A. M. Salih *et al*, "A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME," *Advanced Intelligent Systems*, vol. 7, no. 1, Jan. 2025, doi: 10.1002/aisy.202400304.
- [17] P. Naga Srinivasu, M. F. Ijaz, and M. Woźniak, "XAI-driven model for crop recommender system for use in precision agriculture," *Comput Intell*, vol. 40, no. 1, Feb. 2024, doi: 10.1111/coin.12629.
- [18] K. C. Bodduluri, F. Palma, A. Kurti, I. Jusufi, and H. Lowenadler, "Exploring the Landscape of Hybrid Recommendation Systems in E-Commerce: A Systematic Literature Review," *IEEE Access*, vol. 12, pp. 28273–28296, 2024, doi: 10.1109/ACCESS.2024.3365828.
- [19] F. Ricci, B. Shapira, and L. Rokach, "Recommender systems: Introduction and challenges," in *Recommender Systems Handbook, Second Edition*, Springer US, 2015, pp. 1–34. doi: 10.1007/978-1-4899-7637-6_1.
- [20] U. Javed, K. Shaukat, I. A. Hameed, F. Iqbal, T. M. Alam, and S. Luo, "A Review of Content-Based and Context-Based Recommendation Systems," *International Journal of Emerging Technologies in Learning*, vol. 16, no. 3, pp. 274–306, 2021, doi: 10.3991/ijet.v16i03.18851.